

DOKUMEN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Automata

Arion Compiler : Milestone 3 - Semantic Analysis



Nama Anggota :

Zahran Alvan Putra Winarko	13524124
Neswa Eka Anggara	13524136
Daniel Putra Rywandi S.	13524143
Muh. Hartawan Haidir	13524147

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

BAB I

TEORI SINGKAT

1. Abstract Syntax Tree dan Decorated AST

Abstract Syntax Tree merupakan bentuk yang lebih sederhana dan ringkas dari parse tree. Jika parse tree masih mengikuti seluruh aturan tata bahasa untuk keperluan pengecekan sintaksis, maka Abstract Syntax Tree hanya mempertahankan bagian-bagian penting yang berkaitan langsung dengan struktur logika program. Elemen-elemen struktural yang tidak lagi dibutuhkan dalam tahap analisis lanjutan, seperti tanda titik koma, kata kunci BEGIN dan END, maupun tanda kurung yang berlebihan, akan dihilangkan. Dengan demikian, Abstract Syntax Tree lebih berfokus pada hubungan antara operator dan operand dalam program.

Pada tahap Semantic Analysis, Abstract Syntax Tree kemudian diberi anotasi sehingga menghasilkan Decorated Abstract Syntax Tree. Proses anotasi ini dilakukan dengan menambahkan berbagai informasi penting ke setiap node, seperti tipe data hasil inferensi, referensi memori, tingkat lexical level, serta indeks identifier yang terhubung dengan tabel simbol. Informasi tambahan tersebut digunakan untuk membantu proses validasi semantik dan tahap kompilasi berikutnya.

2. Attributed Grammar dan Visitor Pattern

Analisis semantik diimplementasikan menggunakan konsep L-Attributed Grammar, yaitu mekanisme evaluasi atribut yang dilakukan secara top-down dengan metode depth-first search. Dalam proses ini, setiap node pada pohon sintaks akan dievaluasi dan dianotasi sesuai aturan atribut yang dimiliki. Untuk mempermudah penelusuran dan pengolahan node, digunakan pendekatan Visitor Pattern, yaitu penerapan fungsi penelusur khusus (visitor functions) untuk setiap jenis node produksi. Pendekatan ini membuat proses analisis semantik menjadi lebih terstruktur, modular, dan mudah dikembangkan.

BAB II

PERANCANGAN & IMPLEMENTASI

1. Perancangan Tabel Simbol

- Tabel Identifikasi (tab): Menyimpan detail informasi mengenai seluruh nama pengenalan (konstanta, variabel, tipe, prosedur, atau fungsi). Atribut minimal yang disimpan meliputi nama, jenis objek (obj), jenis tipe dasar (type), referensi komposit (ref), penanda parameter (nrm), level leksikal (lev), alamat memori (adr), dan rantai tautan cakupan (link).
- Tabel Blok (btab): Menyimpan informasi blok cakupan prosedur, fungsi, atau definisi tipe rekaman (record). Menyimpan indeks penunjuk deklarasi terakhir (last), parameter terakhir (lpar), total ukuran parameter (psze), dan total ukuran variabel lokal (vsze).
- Tabel Larik (atab): Menyimpan spesifikasi dimensi larik (array), meliputi tipe indeks (xtyp), tipe elemen (etyp), batas bawah (low), batas atas (high), ukuran elemen (elsz), dan total ukuran memori larik (size).

2. Alur Kerja Program

Alur kerja dari Compiler Arion dibagi menjadi beberapa tahapan, mulai dari penerimaan input hingga penghasilan output berupa *Decorated Abstract Syntax Tree* (AST) dan *Symbol Table*, atau daftar pesan *error* jika ditemukan adanya ketidaksesuaian atau pelanggaran semantik. Berdasarkan keseluruhan program yang telah dibuat, secara rinci langkah-langkahnya adalah seperti berikut:

1) *Input*

Ketika memulai menjalankan program, pengguna akan diberikan pilihan untuk memilih jenis operasi yang akan dilakukan berdasarkan file yang di-input. Pilihan yang diberikan adalah mode *source code* dan *parse tree*.

a. Mode *source code*

Pengguna akan diminta untuk menginput file .txt berisikan *source code*. Setelahnya program akan memanggil Lexer untuk mengubah *source code* menjadi token. Dari token tersebut kemudian diubah menjadi *parse tree* oleh

modul Parser. Baru nantinya *parse tree* tersebut akan diproses oleh Semantic Analyzer.

b. Mode *parse tree*

Pengguna akan diminta untuk menginput file .txt berisikan *parse tree*. *Parse tree* tersebut kemudian akan diproses oleh Semantic Analyzer.

2) Analisis Semantik dan Pembangunan *Symbol Table*

Proses yang dilakukan oleh Semantic Analyzer berjalan secara Top-Down untuk navigasi dan Bottom-Up untuk propagasi tipe data. Program akan menelusuri <program-header> dan <declaration-part>. Setiap variabel, konstanta, array, dan subprogram yang ditemukan akan dicatat ke dalam memori, yaitu *Symbol Table* (tab), *Block Table* (btab), dan *Array Table* (atab) untuk tipe data *array*. Ketika mendapatkan <compound-statement>, program akan membaca <statement-list>. Program kemudian memanggil fungsi *visitStatement* untuk mengidentifikasi jenis pernyataan yang sesuai. Sebagai contoh, jika menemukan If, maka akan dipanggil fungsi *visitIfStatement*.

3) Validasi dan *Type Checking*

Fungsi seperti *visitIfStatement* dan *visitWhileStatement* memeriksa apakah ekspresi evaluasi memiliki *typeCode* berupa Boolean. Untuk for, akan diperiksa apakah *counter* dan rentangnya berupa integer (harus integer). Fungsi *visitAssignStatement* membandingkan tipe data target dengan tipe data dari *value*-nya. Jika tidak sesuai, misalnya sebuah variabel integer dimasukkan string, program akan mencatat error tersebut.

4) *Error Handling*

Compiler dilengkapi dengan mekanisme error handling untuk menangani kesalahan semantic tanpa membuat program menjadi hancur atau crash, Untuk semantic error yang dapat dideteksi adalah

1. Penggunaan variabel yang belum dideklarasikan
2. Deklarasi ulang tentang identifier dalam jangkauan atau scope yang sama

3. Assignment incompatible
4. Penggunaan operator yang tidak sesuai dengan tipenya
5. Parameter prosedur / fungsi yang tidak sesuai

Pesan Error akan ditampilkan secara penuh dan informatif sehingga mempermudah proses debugging.

5) *Output*

Output yang akan dihasilkan oleh program berupa beberapa keluaran utama, yaitu :

1. *Decorated Abstract Syntax tree*
2. Simbol Tabel (`tab`, `btabs`, `atab`)
3. Pesan semantic error apabila ditemukan kesalahan dalam program

Selain itu, ditemukan juga beberapa informasi tambahan terkait dengan *Decorated AST*, yaitu :

- Tipe data node
- Lexical level
- Indeks simbol tabel
- Referensi identifier

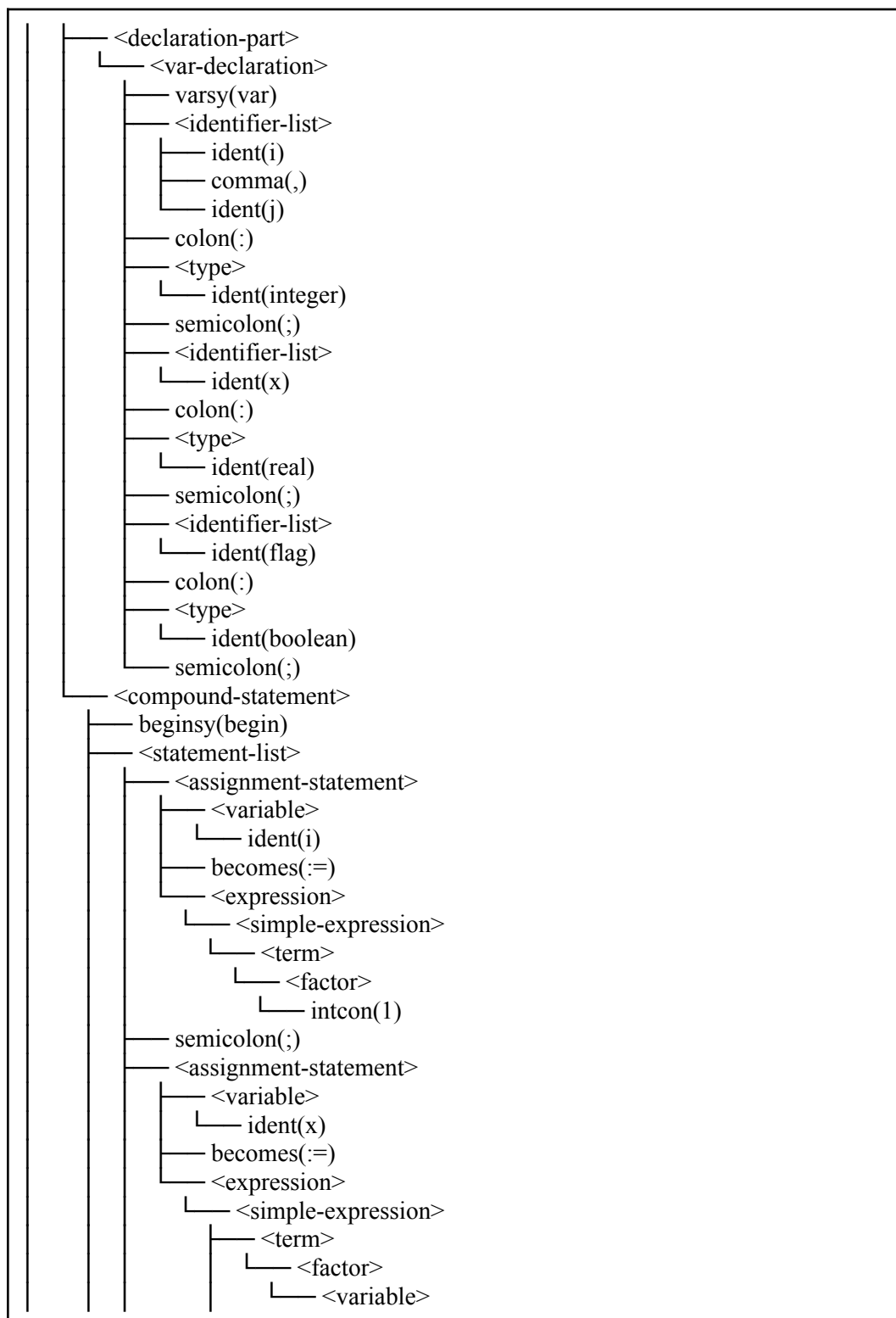
BAB III

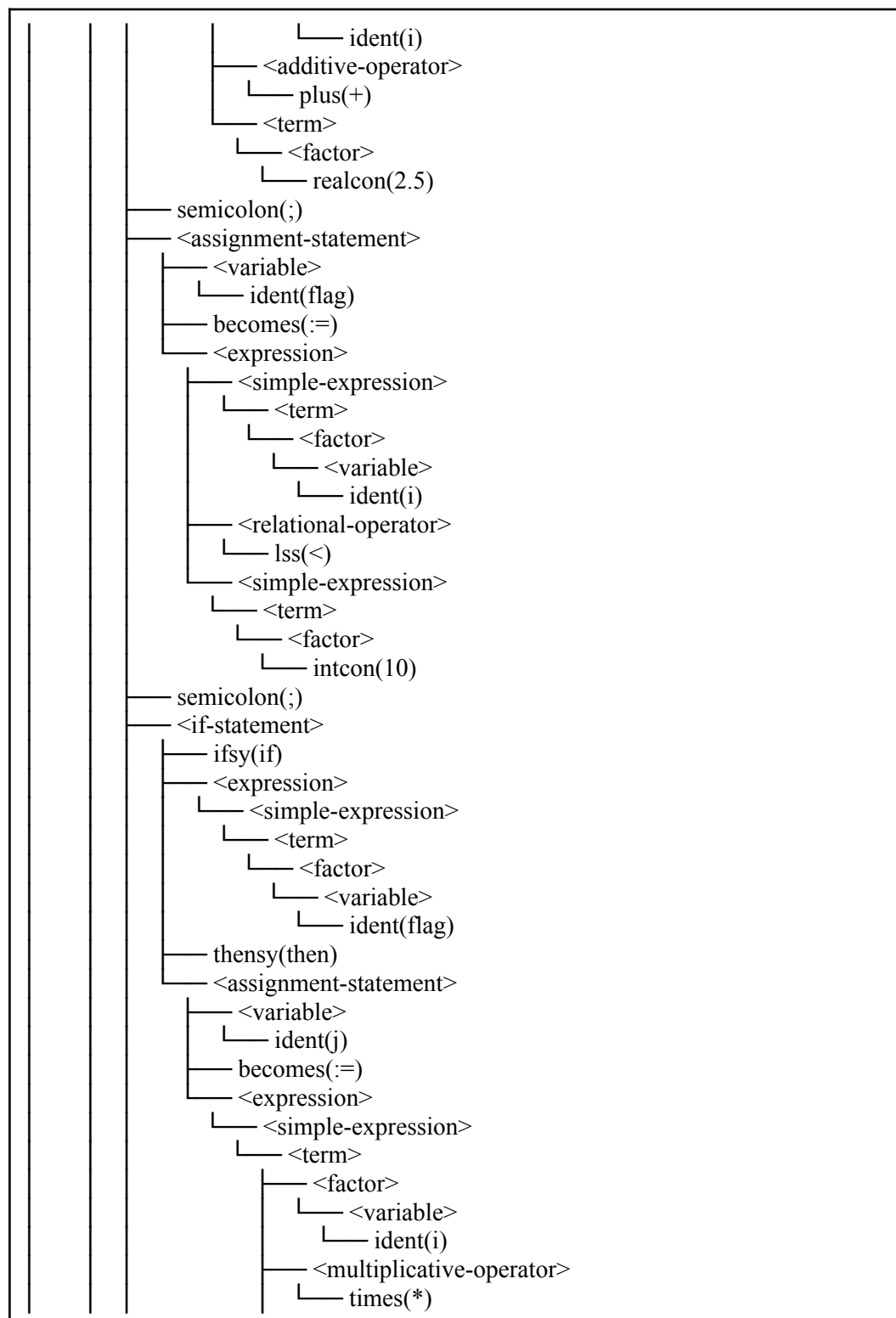
PENGUJIAN

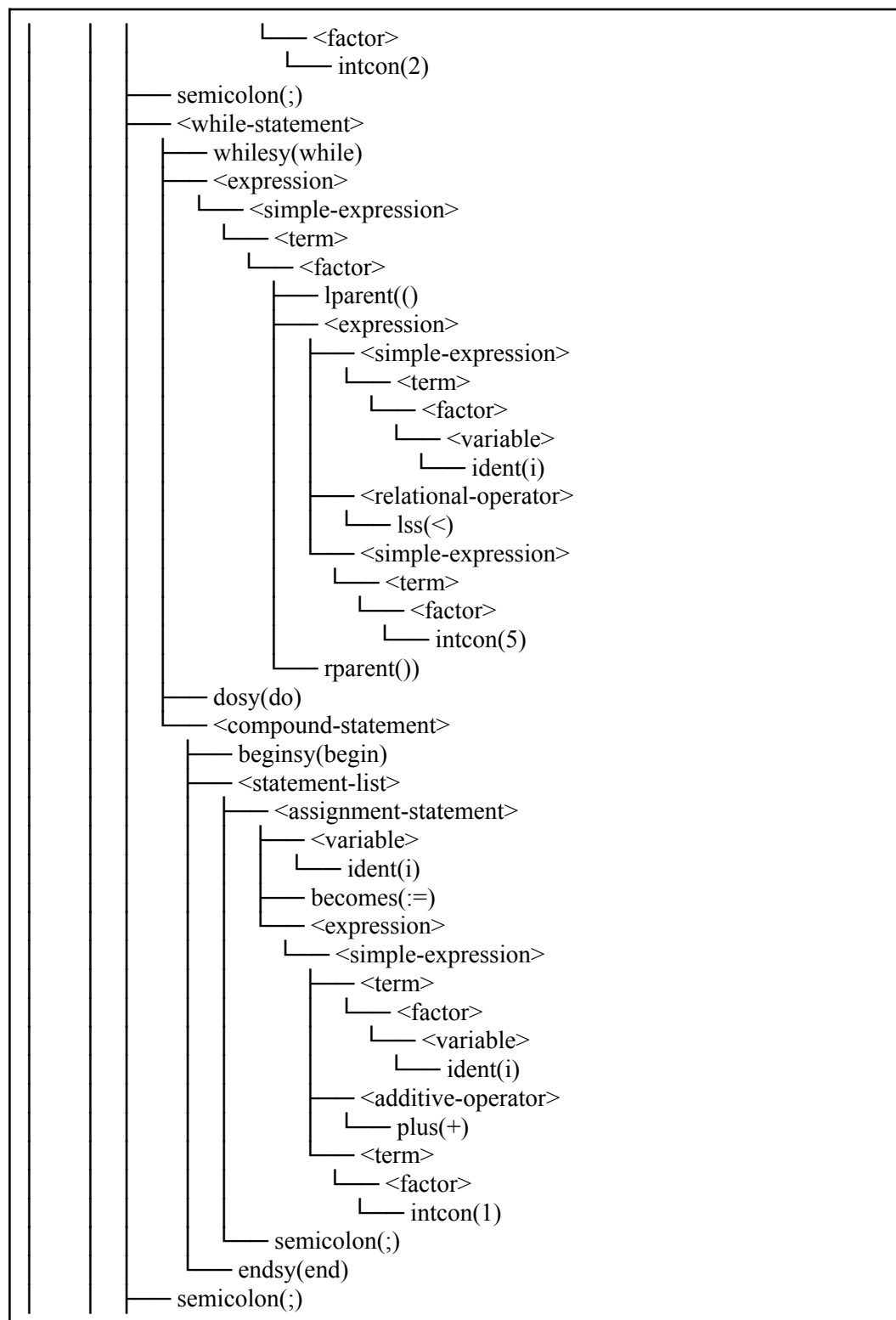
1. Test Case 1

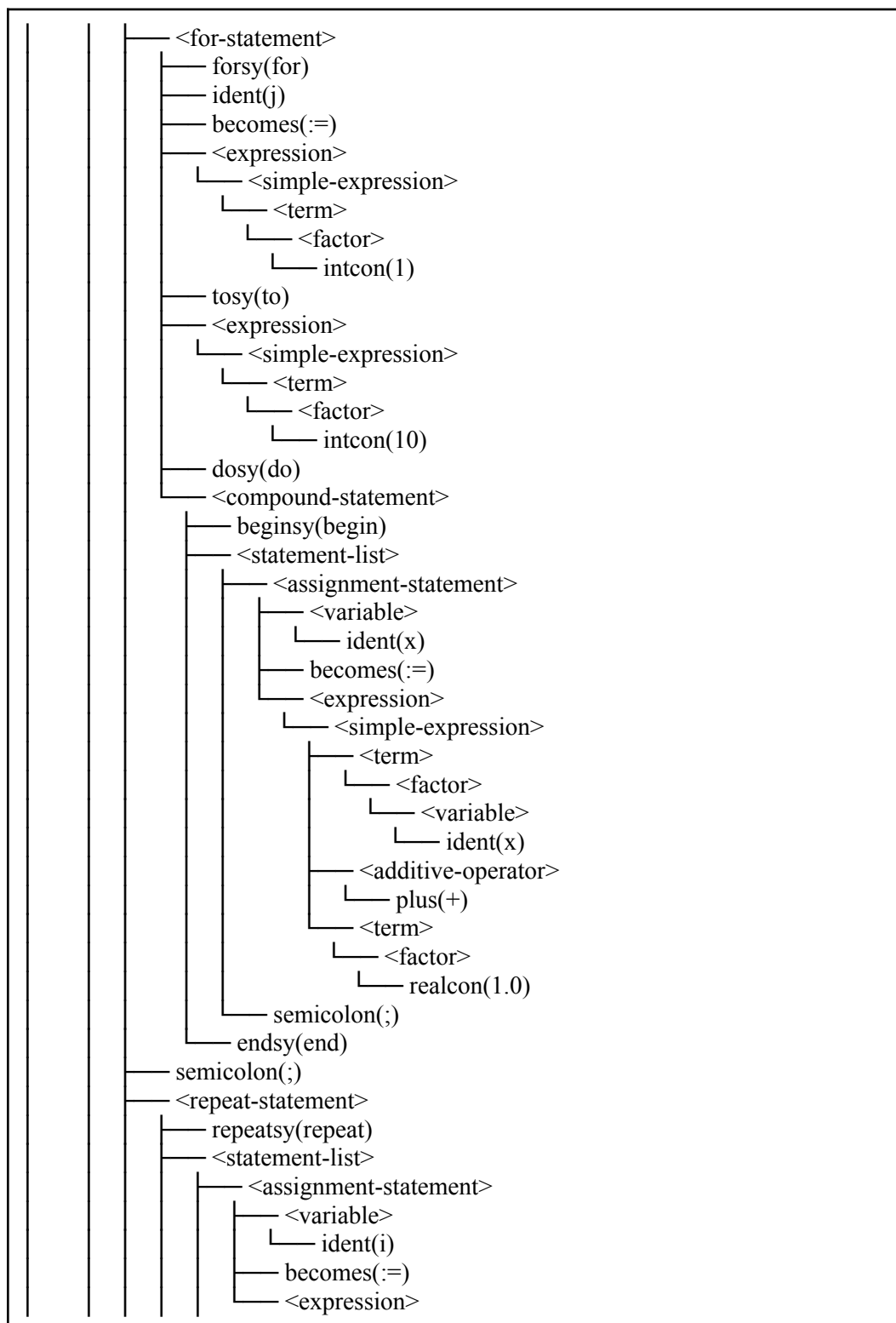
1.1. Input:

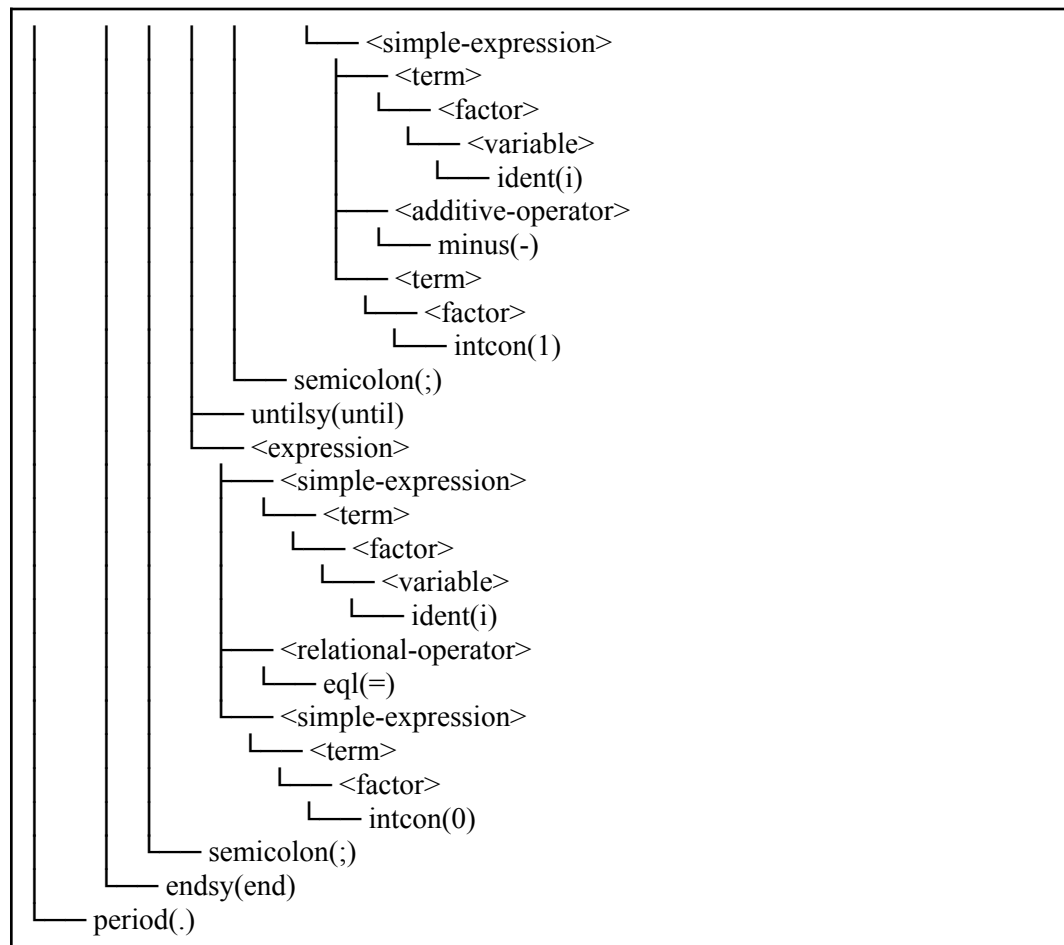
```
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(TestValid)
│   └── semicolon(;)
└── <block>
```



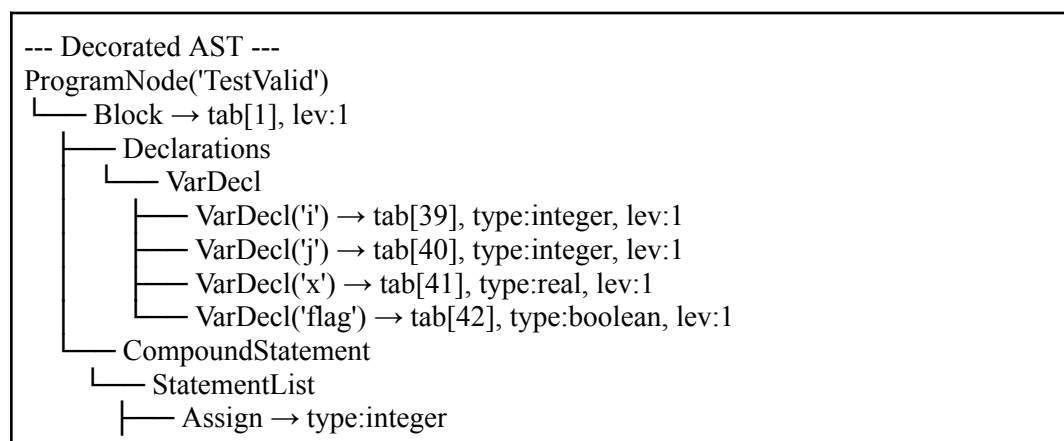


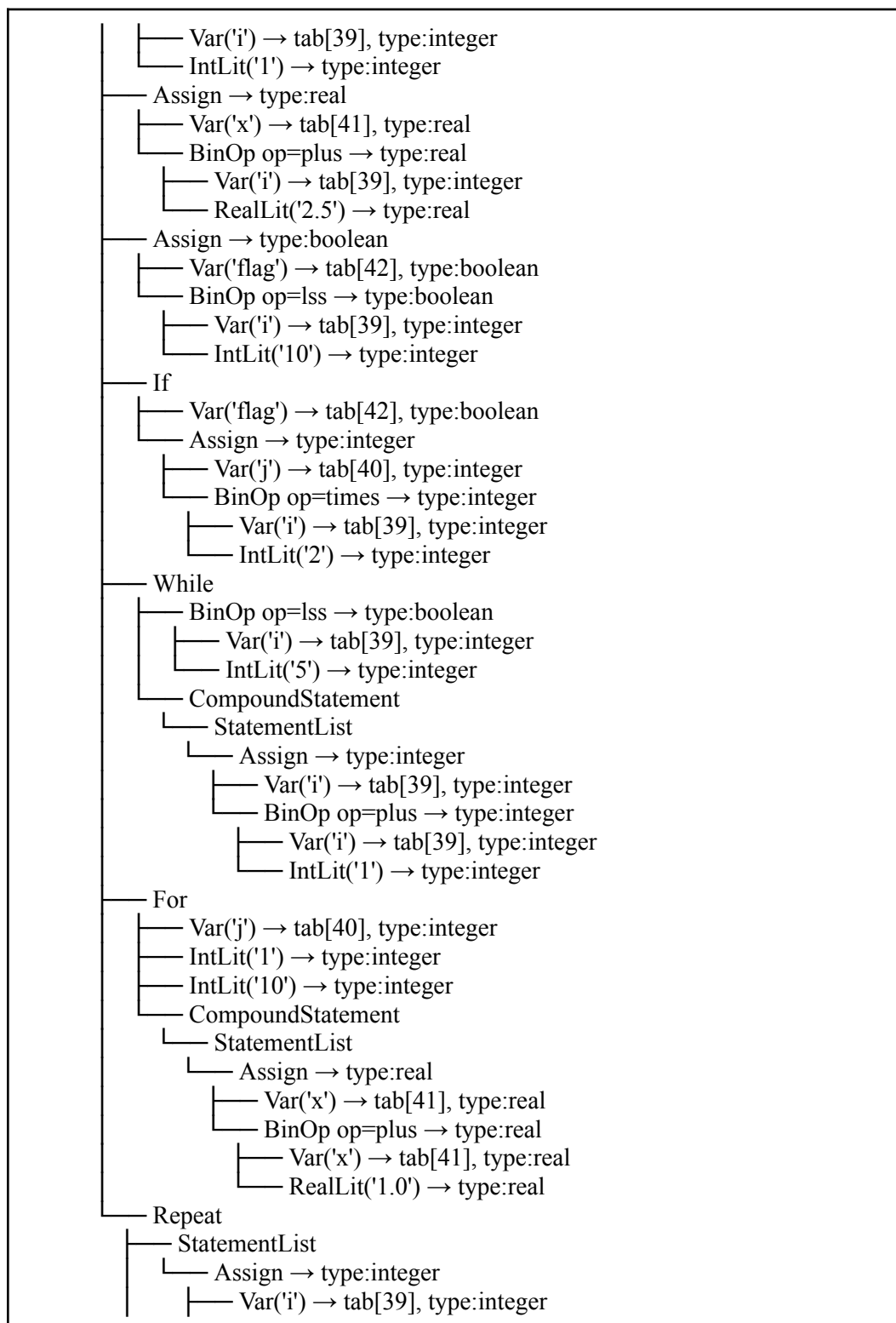


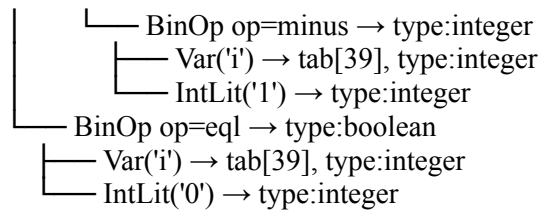




1.2. Output:







--- Symbol Table (tab) ---

idx	identifier	obj	type	ref	nrm	lev	adr	link
38	testvalid	program	none	0	1	0	0	37
39	i	variable	integer	0	1	1	0	0
40	j	variable	integer	0	1	1	1	39
41	x	variable	real	0	1	1	2	40
42	flag	variable	boolean	0	1	1	3	41

--- Block Table (btab) ---

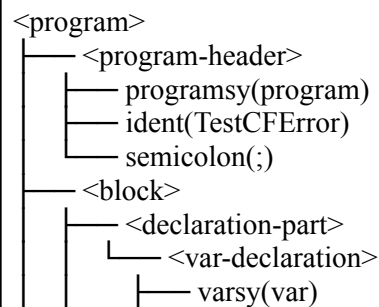
idx	last	lpar	psze	vsze
0	38	0	0	0
1	42	0	0	4

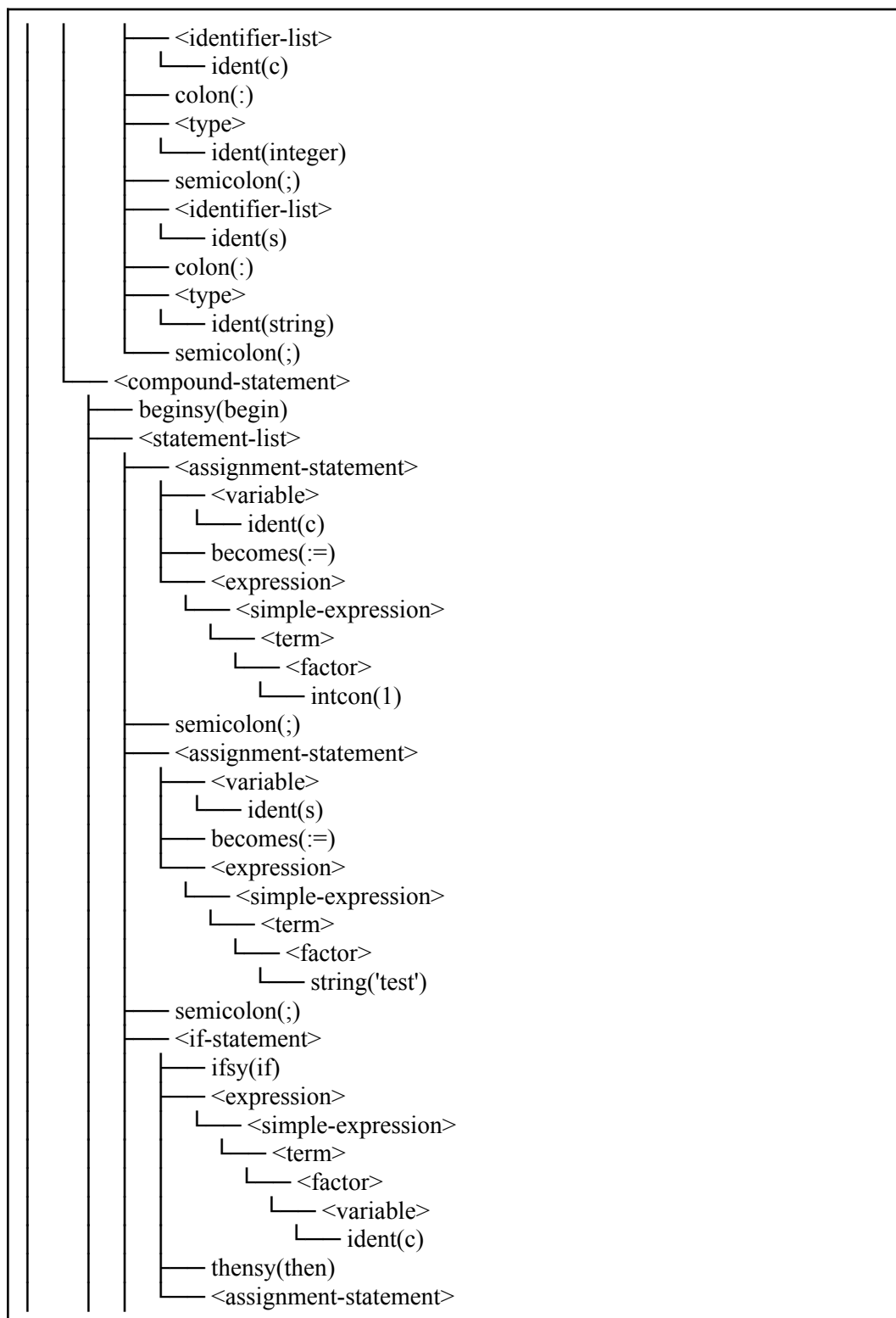
--- Array Table (atab) ---

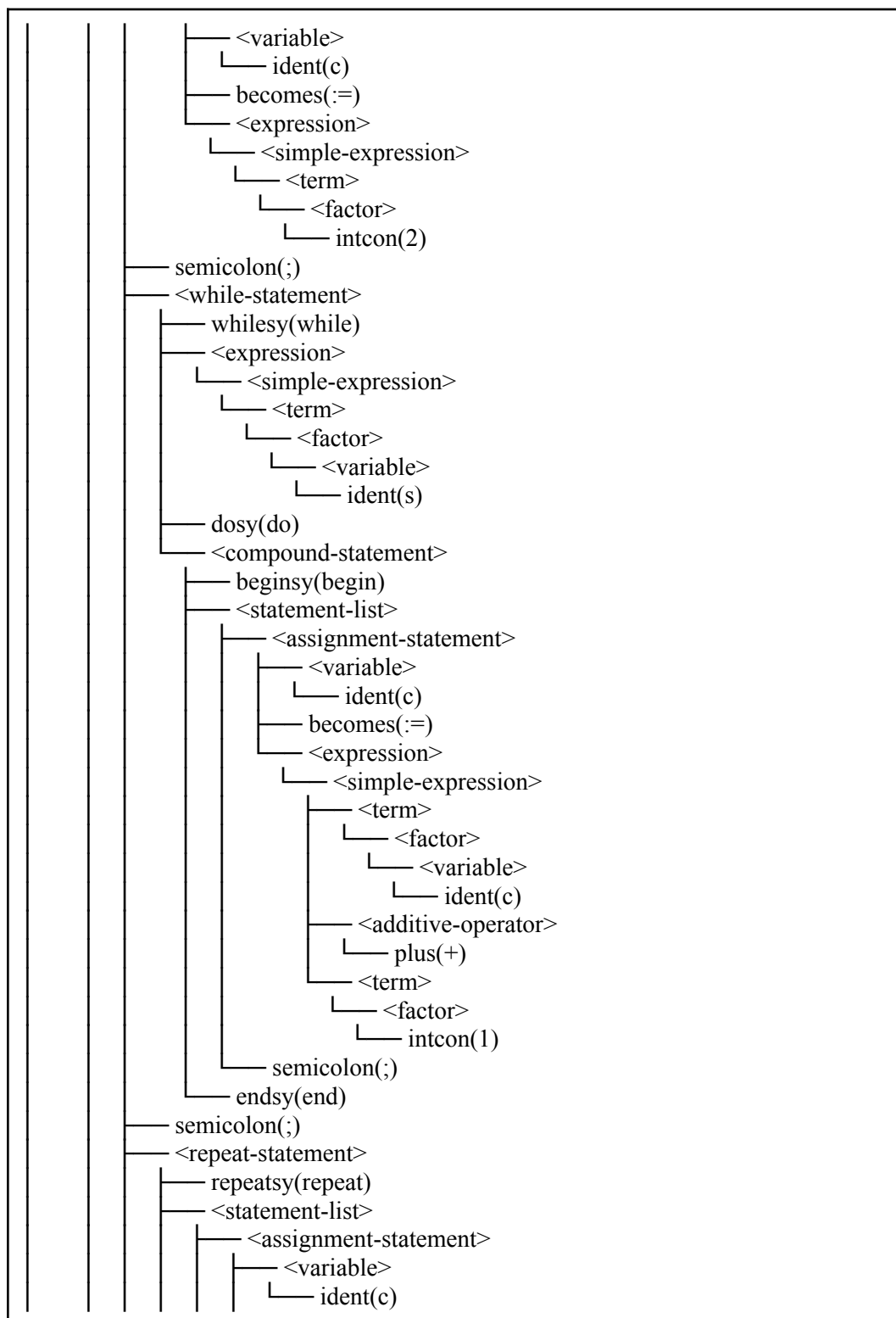
(kosong)

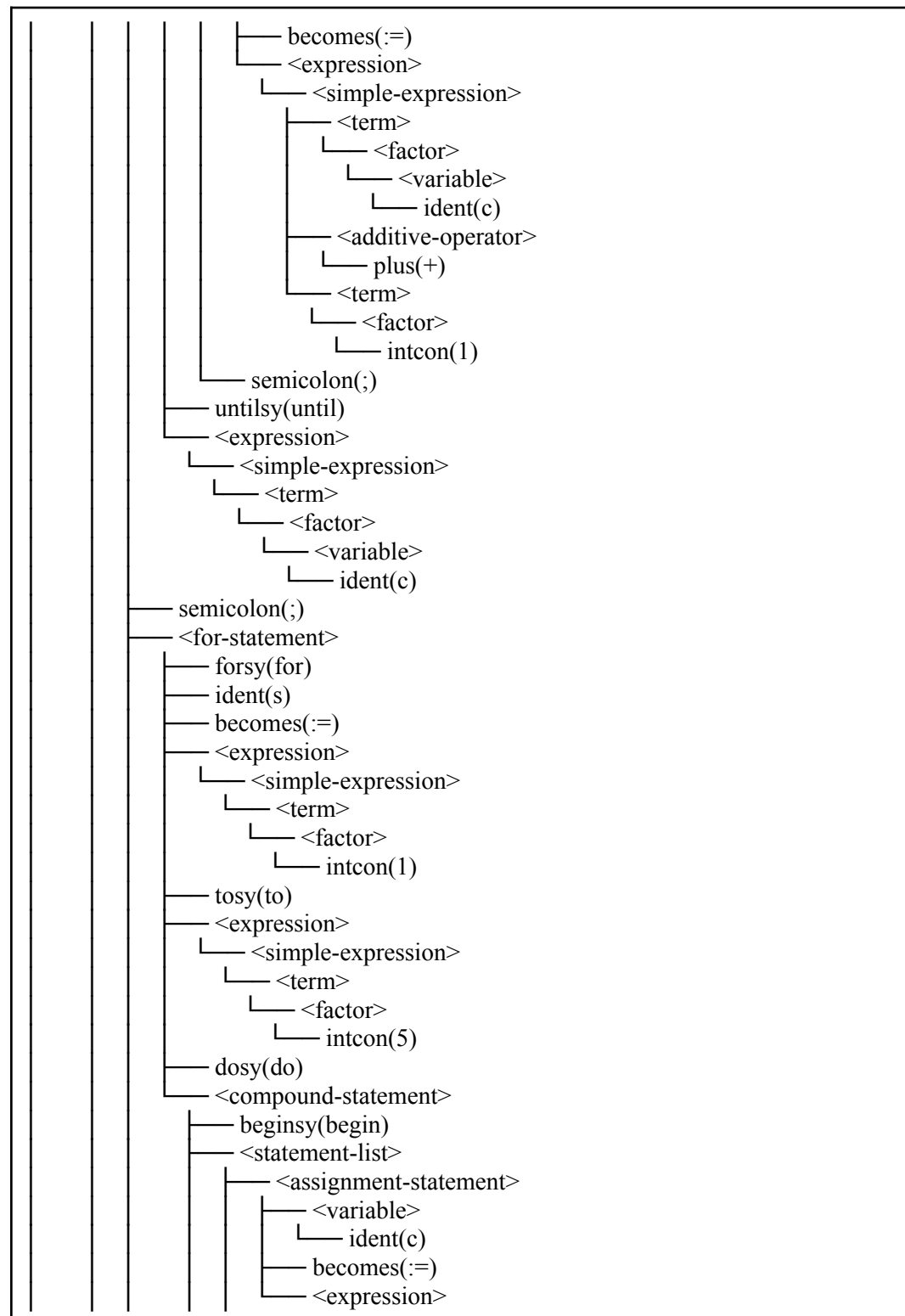
2. Test Case 2

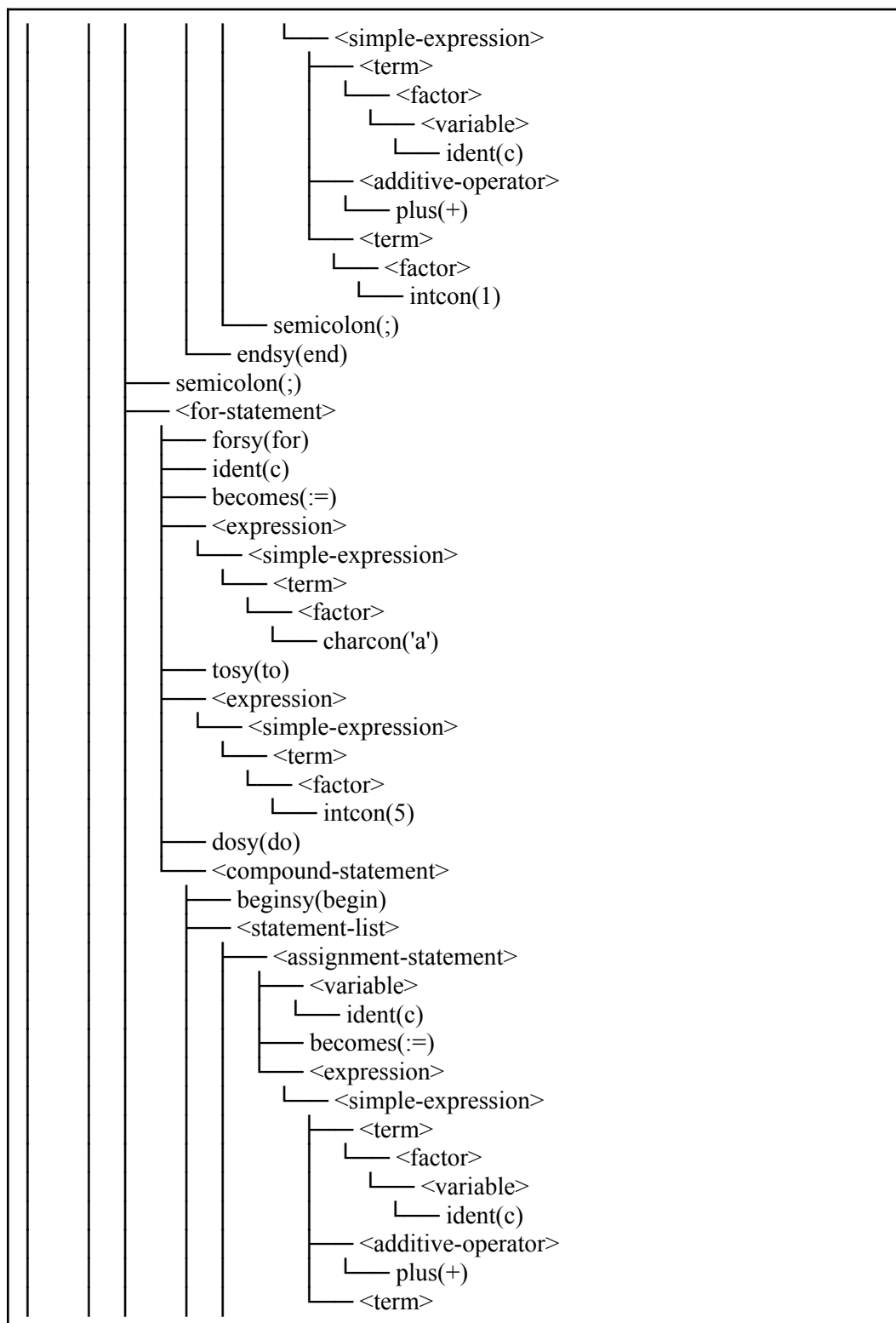
2.1. Input:

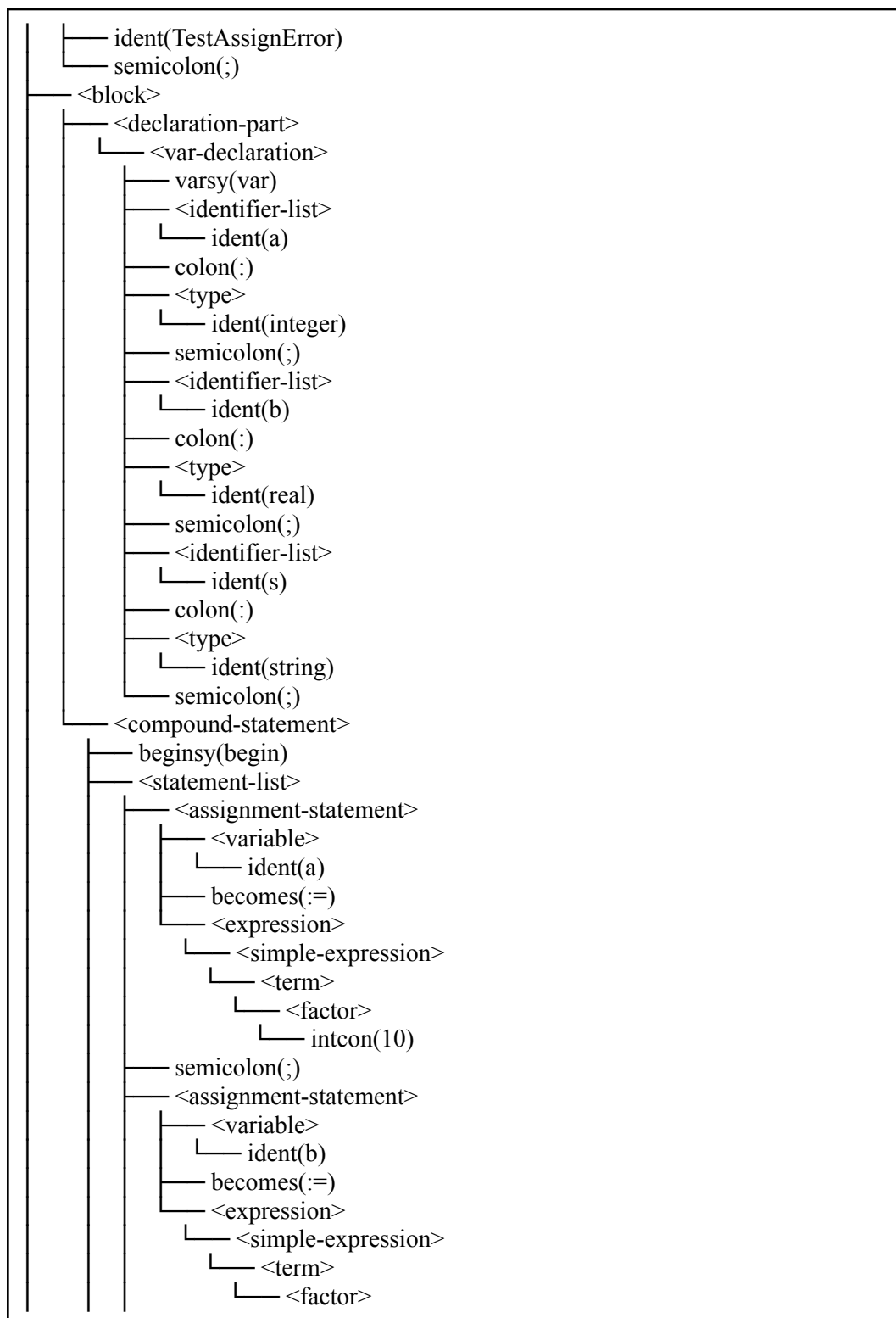


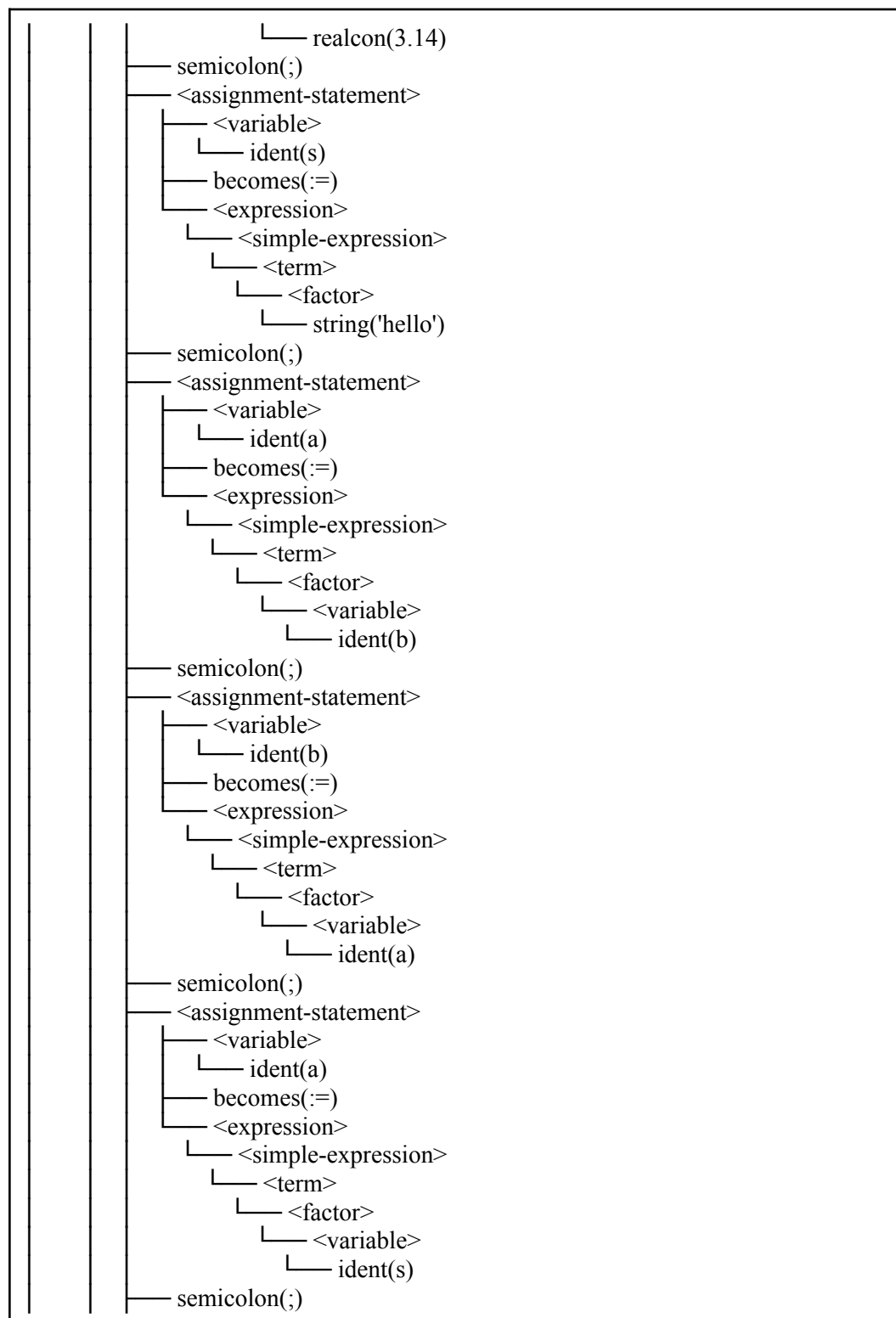


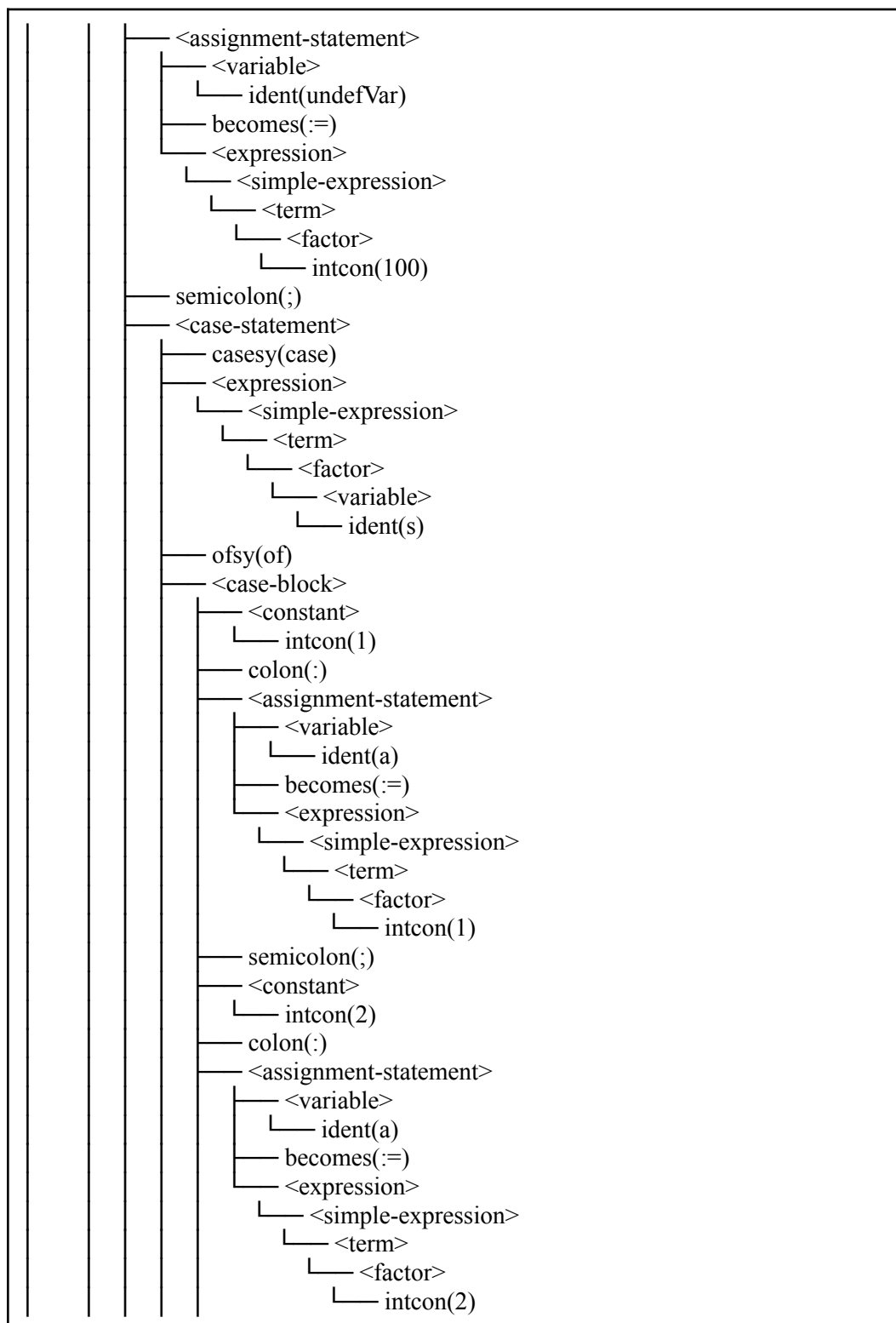


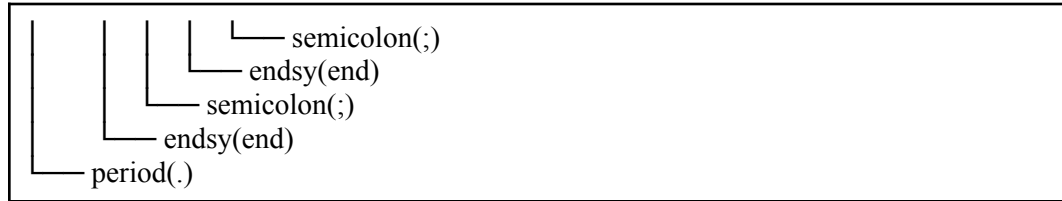












3.2. Output:

```

--- Semantic Errors (6) ---
- Semantic Error: Type mismatch in assignment: cannot assign 'real' to 'integer'
- Semantic Error: Type mismatch in assignment: cannot assign 'string' to 'integer'
- Semantic Error: Variabel tidak dideklarasikan: undefVar
- Semantic Error: Type mismatch in assignment: cannot assign 'integer' to 'none'
- Semantic Error: Case label constant type mismatch with case expression
- Semantic Error: Case label constant type mismatch with case expression

--- Symbol Table (tab) ---
idx identifier      obj      type  ref  nrm  lev  adr  link
-----
38 testassignerror  program  none   0    1    0    0    37
39 a                variable integer 0    1    1    0    0
40 b                variable  real   0    1    1    1    39
41 s                variable  string 0    1    1    2    40

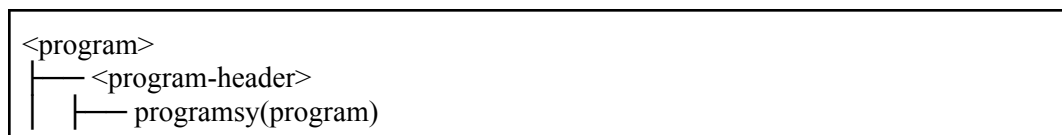
--- Block Table (btab) ---
idx last lpar psze vsze
-----
0  38   0   0   0
1  41   0   0   3

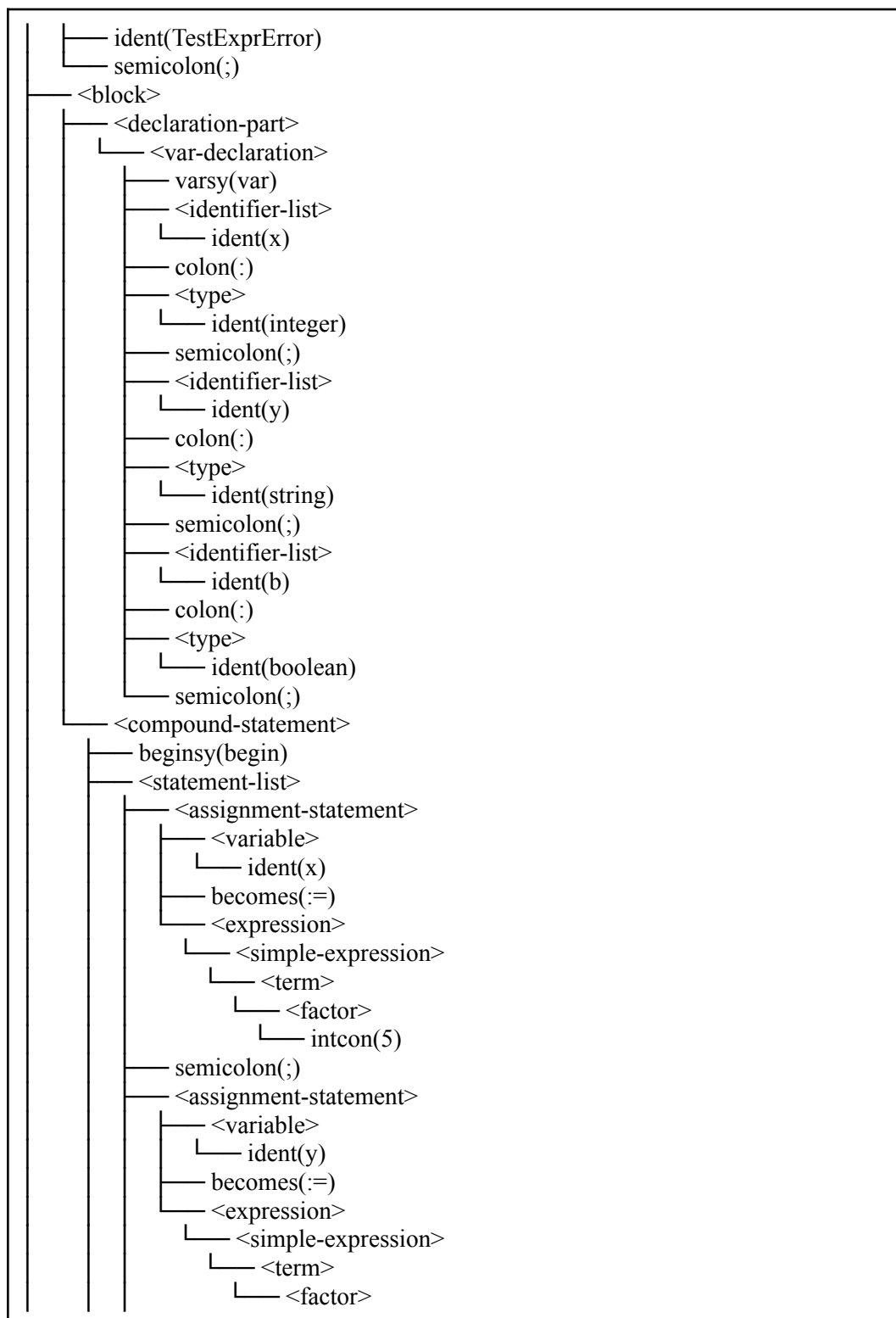
--- Array Table (atab) ---
(kosong)

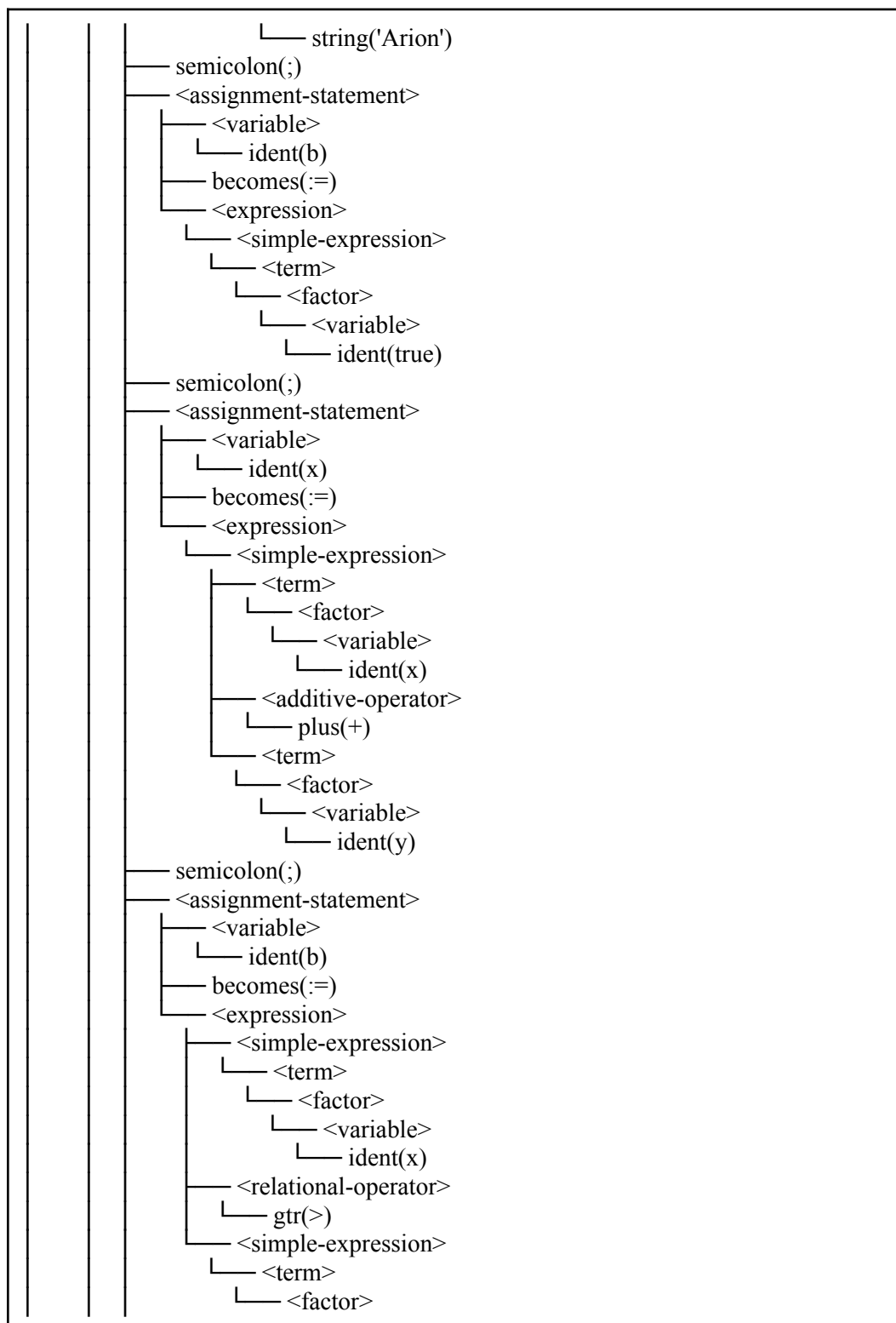
```

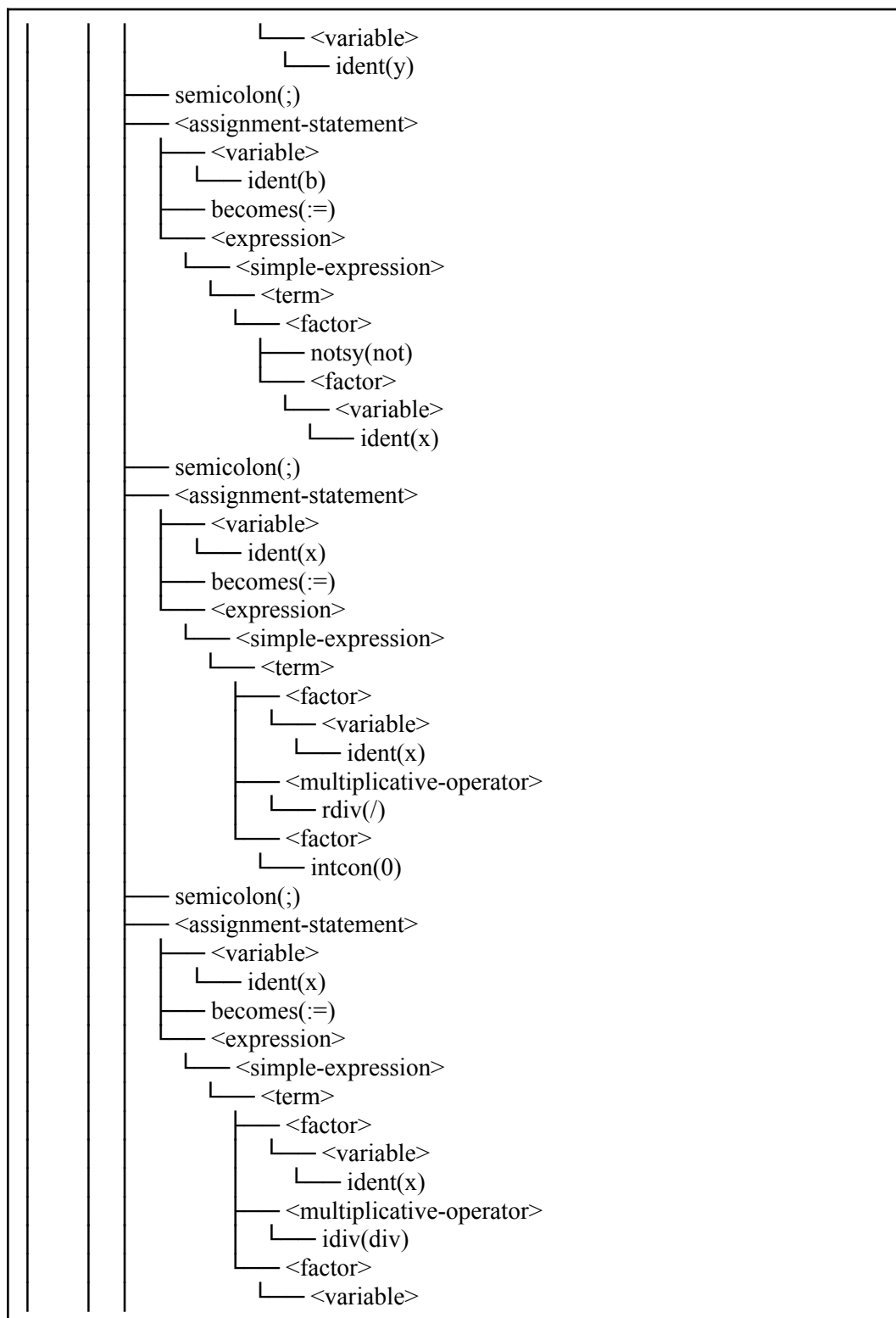
4. Test Case 4

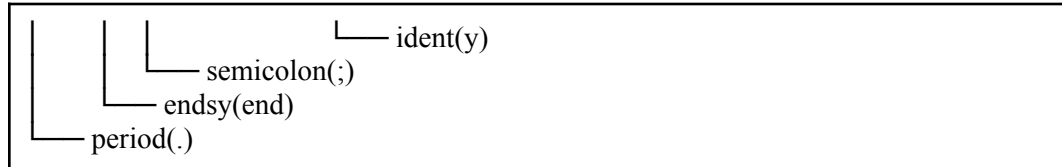
4.1. Input:











4.2. Output:

```

--- Semantic Errors (1) ---
- Semantic Error: Type mismatch in relational operator 'gtr': cannot compare integer
and string

--- Symbol Table (tab) ---
idx identifier      obj      type  ref  nrm  lev  adr  link
-----
38 testexprerror    program  none   0    1    0    0   37
39 x                variable integer 0    1    1    0    0
40 y                variable string 0    1    1    1   39
41 b                variable boolean 0    1    1    2   40

--- Block Table (btab) ---
idx last  lpar  psze  vsze
-----
0   38    0    0    0
1   41    0    0    3

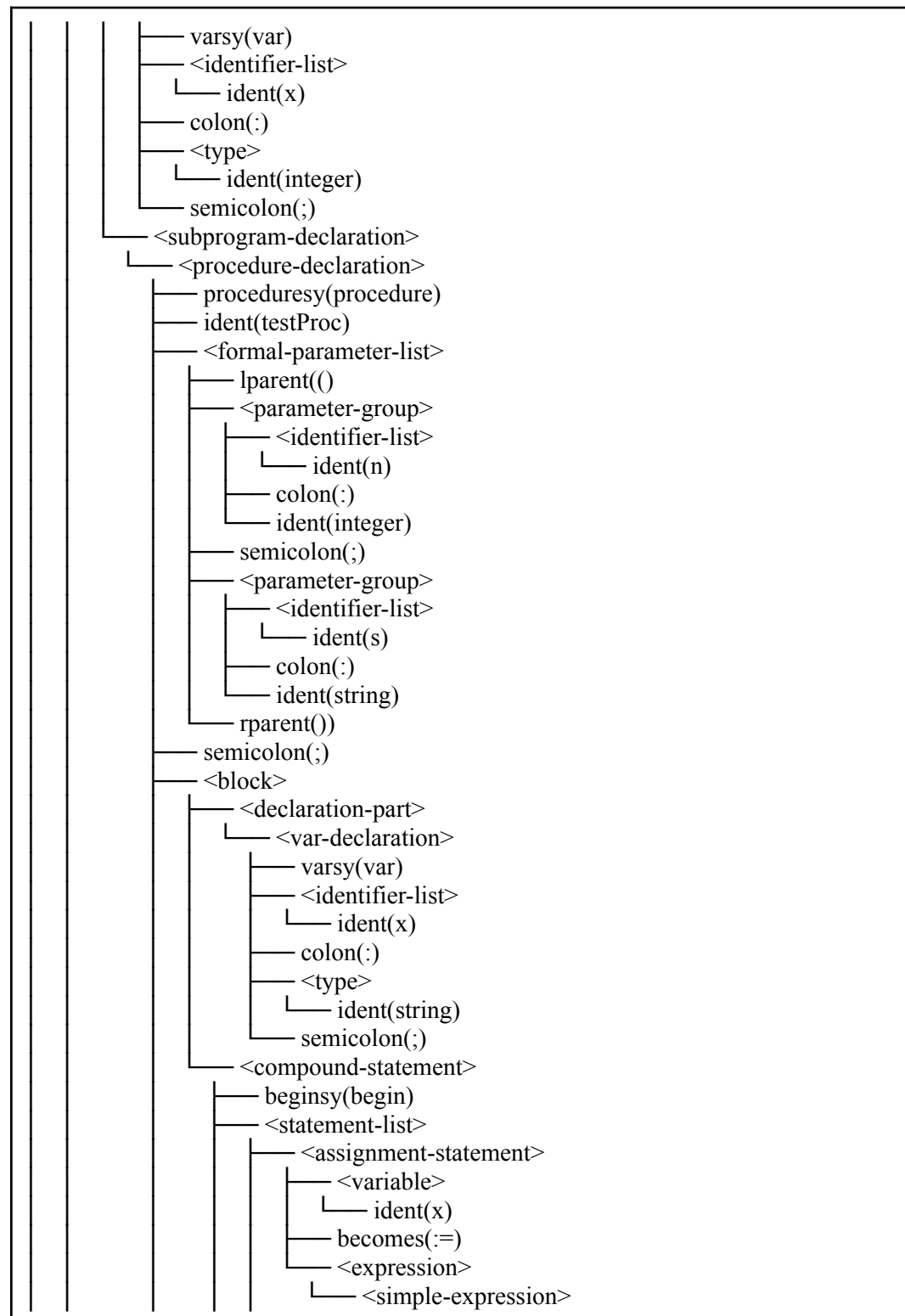
--- Array Table (atab) ---
(kosong)
  
```

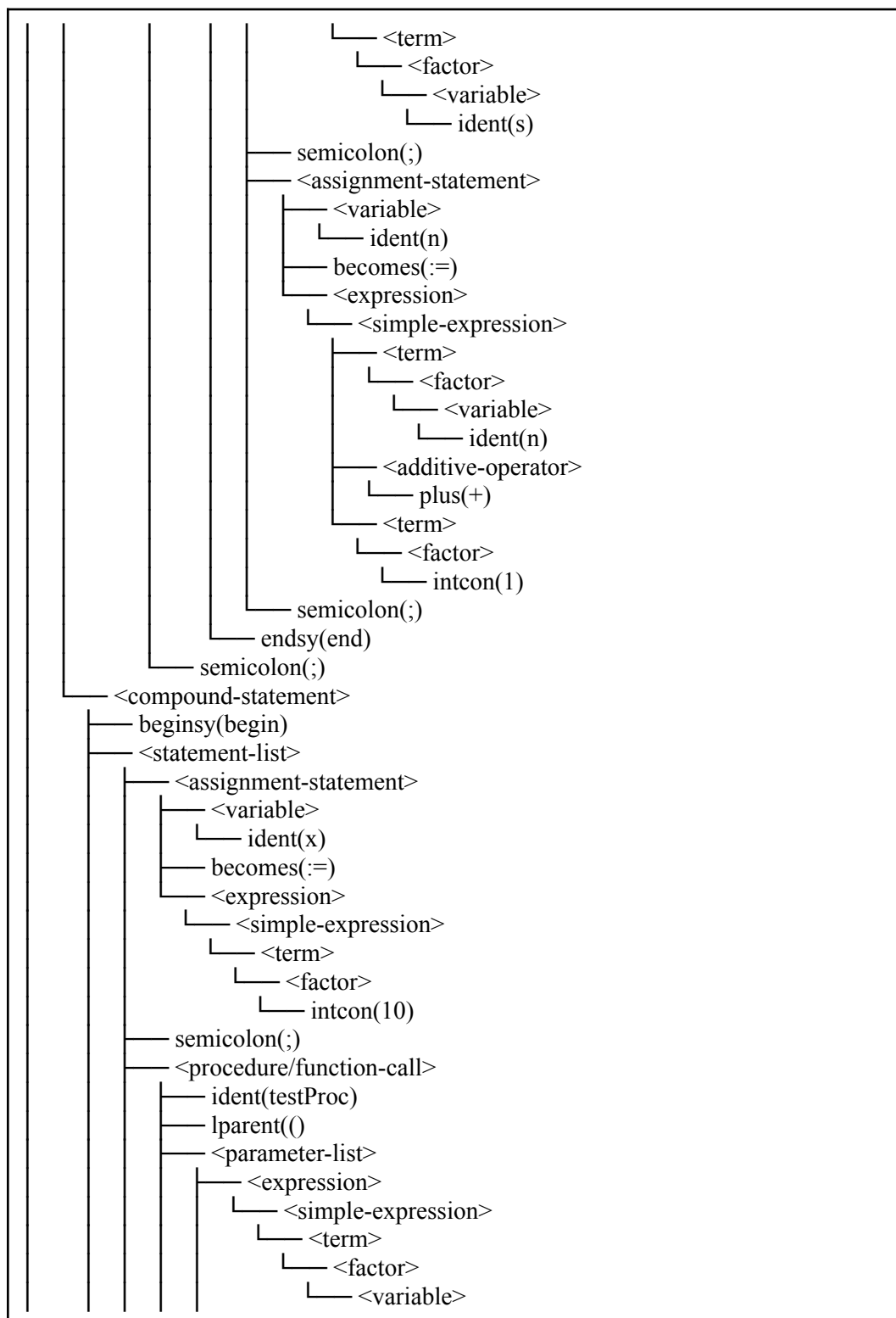
5. Test Case 5

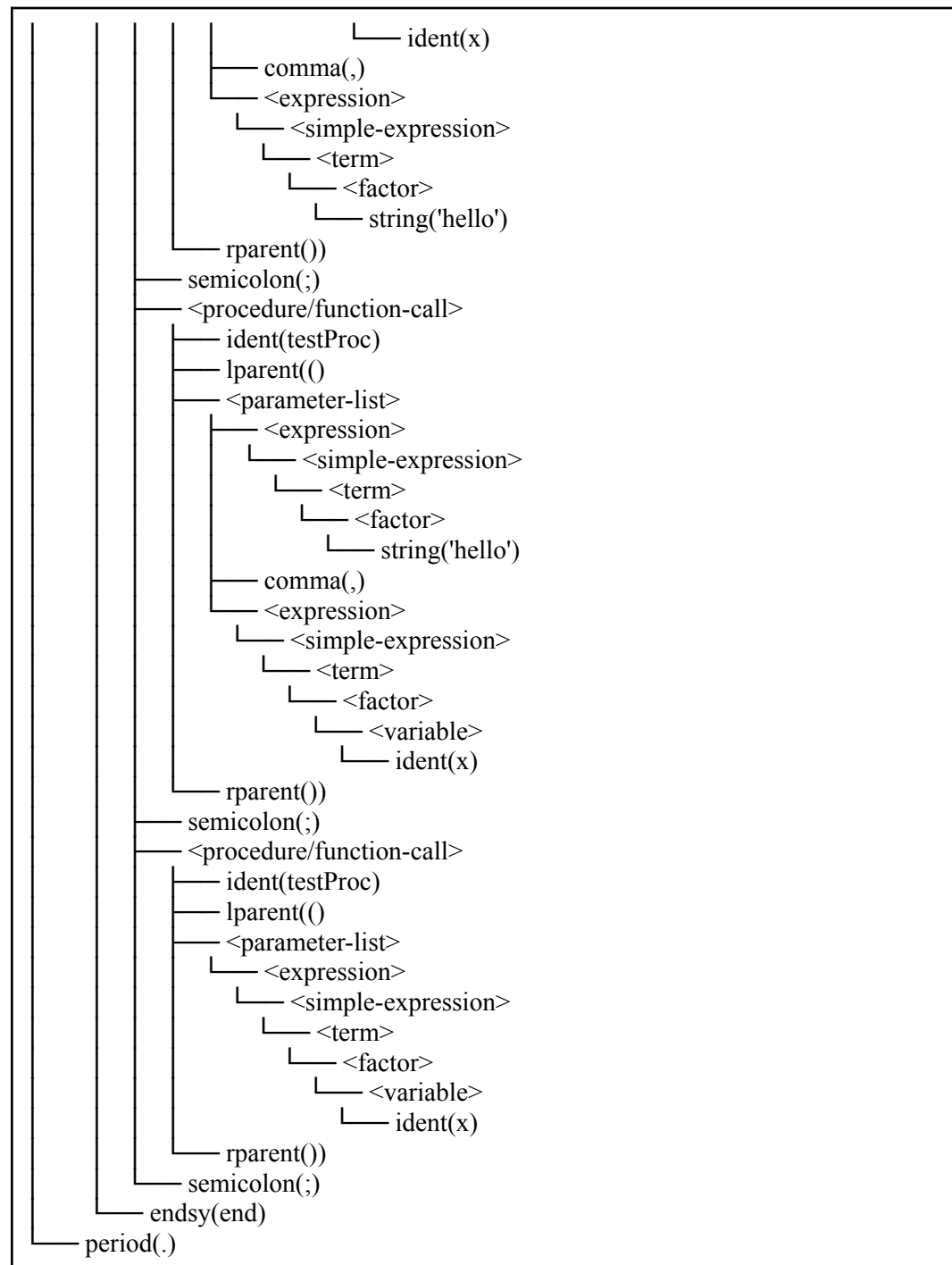
5.1. Input:

```

<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(TestScopeSubprogram)
│   └── semicolon(;)
└── <block>
    ├── <declaration-part>
    │   └── <var-declaration>
    
```



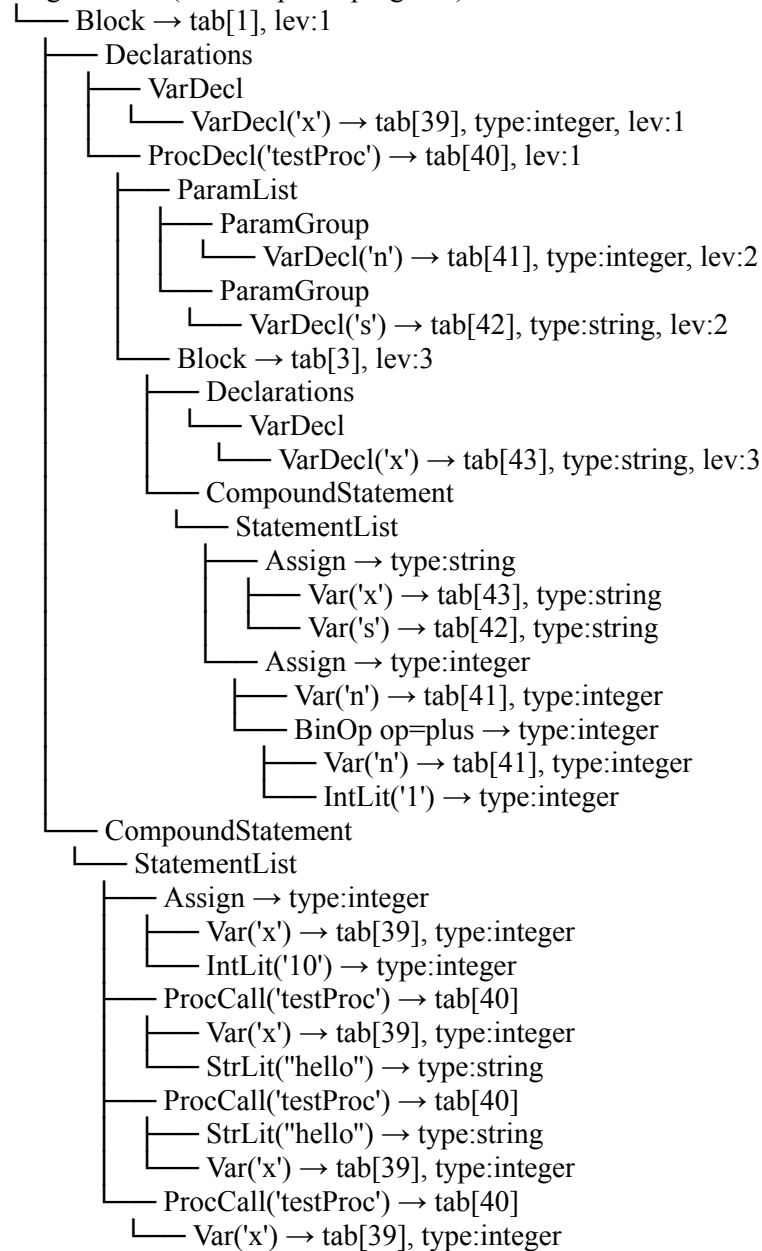




5.2. Output:

--- Decorated AST ---

ProgramNode('TestScopeSubprogram')



--- Symbol Table (tab) ---

idx	identifier	obj	type	ref	nrm	lev	adr	link
38	testscopesubprogram	program	none	0	1	0	0	37

```

39 x          variable integer 0  1  1  0  0
40 testproc   procedure none  2  1  1  0  39
41 n          variable integer 0  1  2  0  0
42 s          variable string  0  1  2  1  41
43 x          variable string  0  1  3  0  0

```

--- Block Table (btab) ---

idx	last	lpar	psze	vsze
0	38	0	0	0
1	40	0	0	1
2	42	42	2	0
3	43	0	0	1

--- Array Table (atab) ---

(kosong)

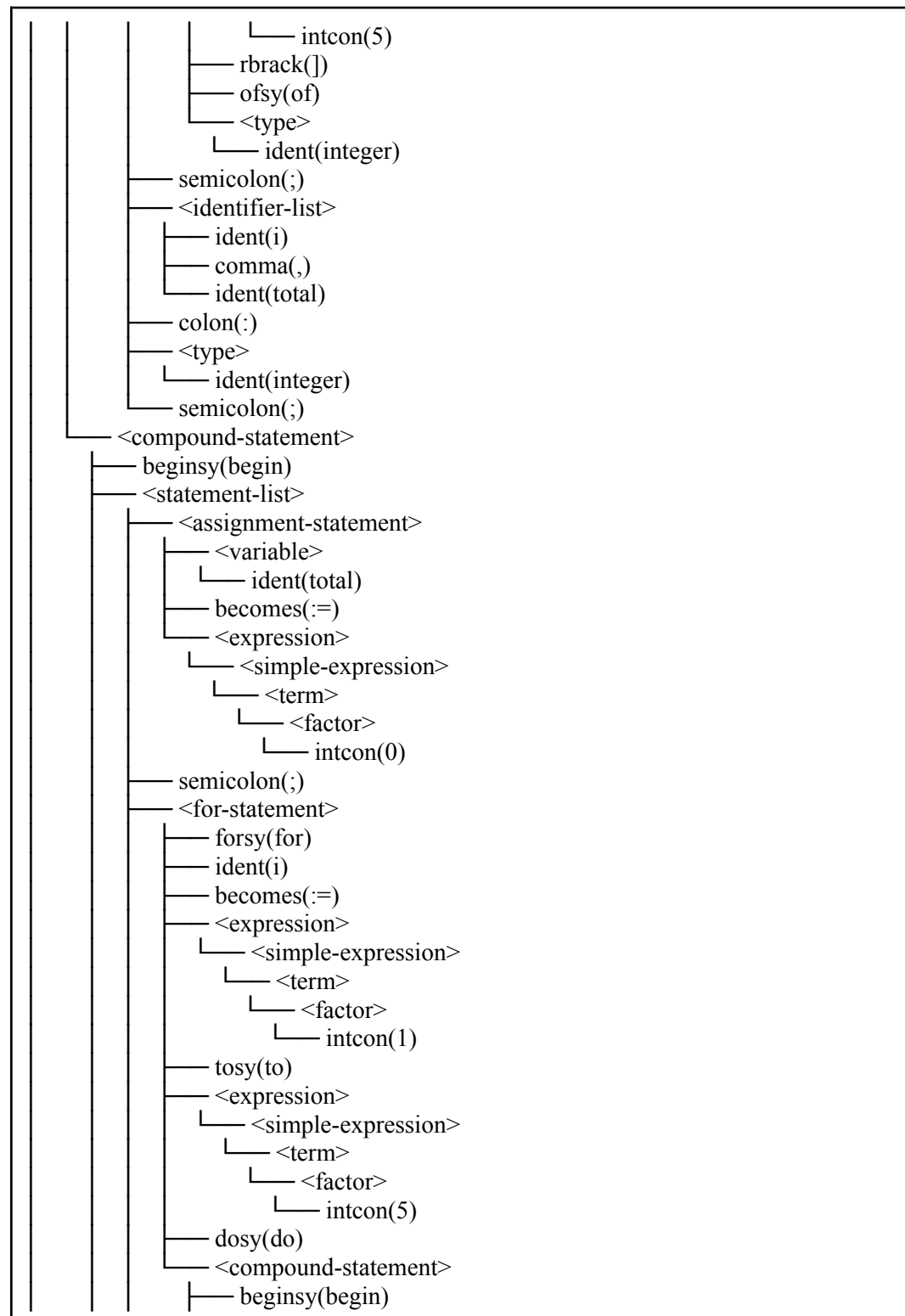
6. Test Case 6

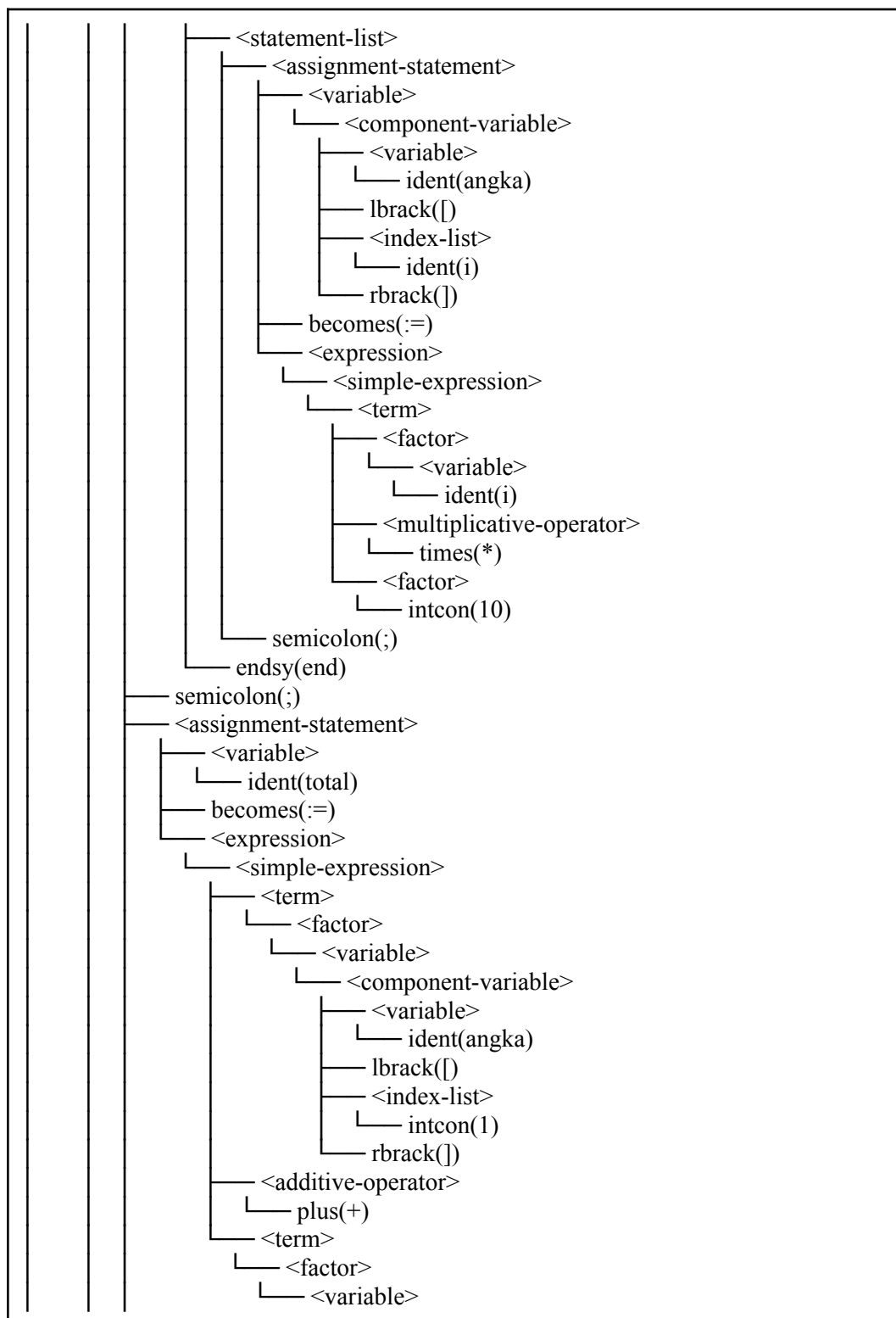
6.1. Input:

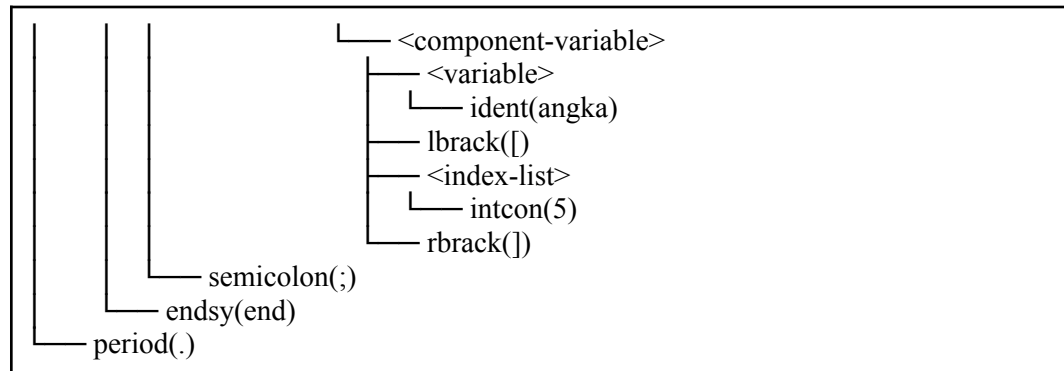
```

<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(TestArrayValid)
│   └── semicolon(;)
├── <block>
│   ├── <declaration-part>
│   │   ├── <var-declaration>
│   │   │   ├── varsy(var)
│   │   │   ├── <identifier-list>
│   │   │   │   ├── ident(angka)
│   │   │   │   ├── colon(:)
│   │   │   └── <type>
│   │   │       ├── <array-type>
│   │   │       │   ├── arraysy(array)
│   │   │       │   ├── lbrack([)
│   │   │       │   └── <range>
│   │   │           ├── <constant>
│   │   │           │   ├── intcon(1)
│   │   │           │   ├── period(.)
│   │   │           │   ├── period(.)
│   │   │           └── <constant>

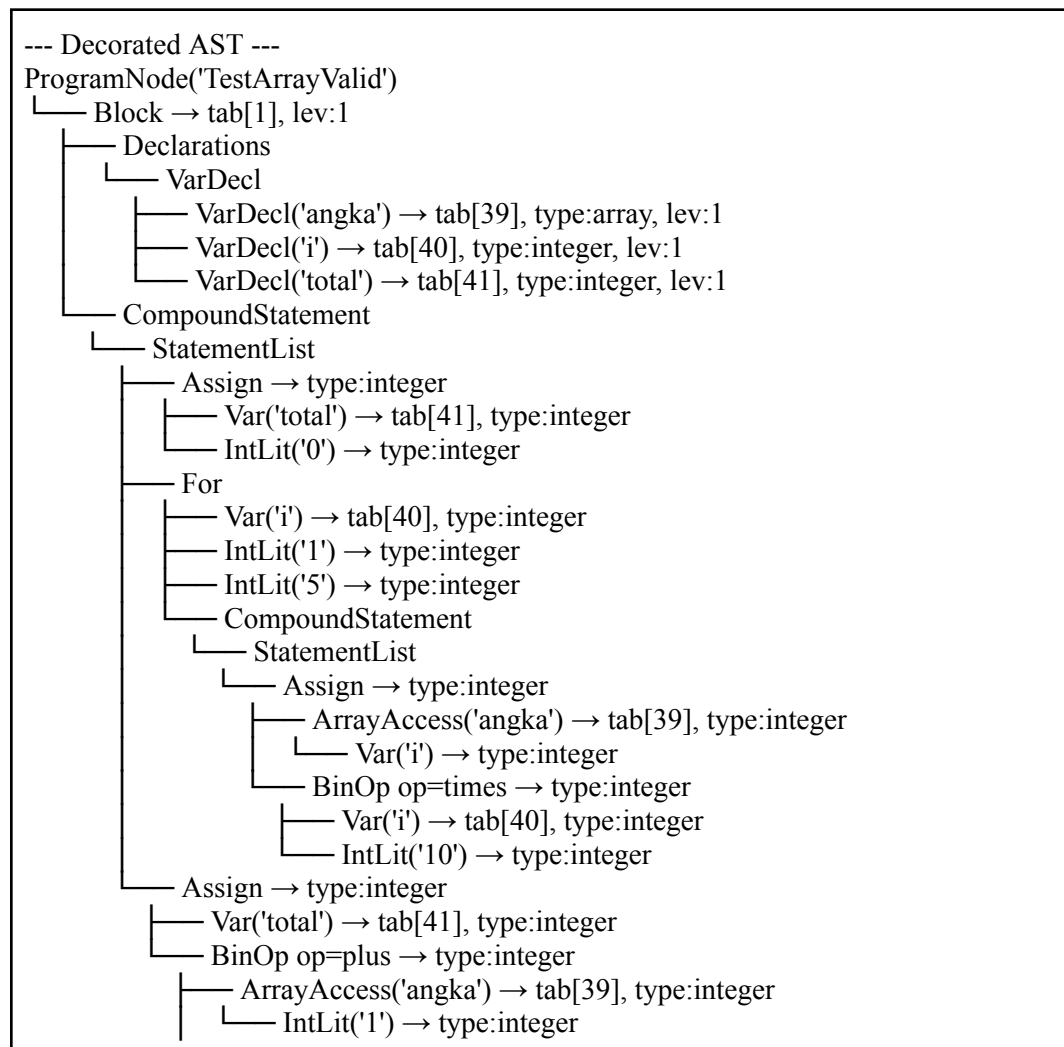
```







6.2. Output:



└─ ArrayAccess('angka') → tab[39], type:integer
 └─ IntLit('5') → type:integer

--- Symbol Table (tab) ---

idx	identifier	obj	type	ref	nrm	lev	adr	link
38	testarrayvalid	program	none	0	1	0	0	37
39	angka	variable	array	1	1	1	0	0
40	i	variable	integer	0	1	1	5	39
41	total	variable	integer	0	1	1	6	40

--- Block Table (btab) ---

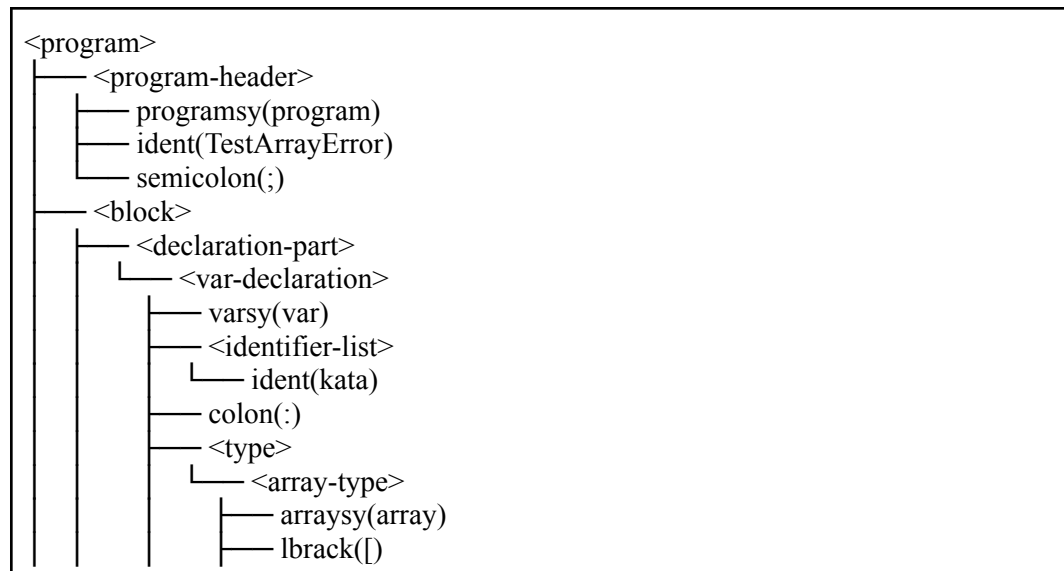
idx	last	lpar	psze	vsze
0	38	0	0	0
1	41	0	0	7

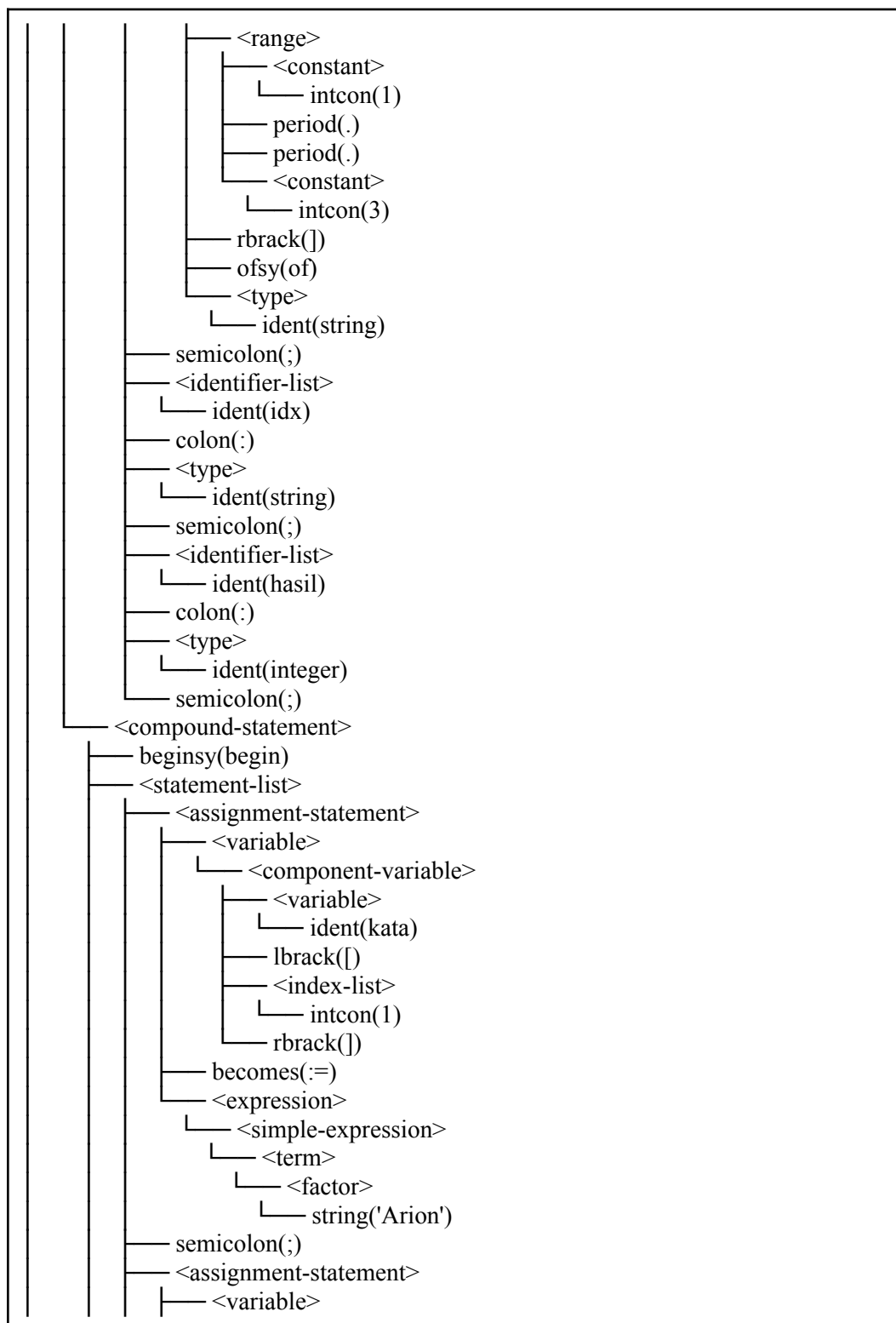
--- Array Table (atab) ---

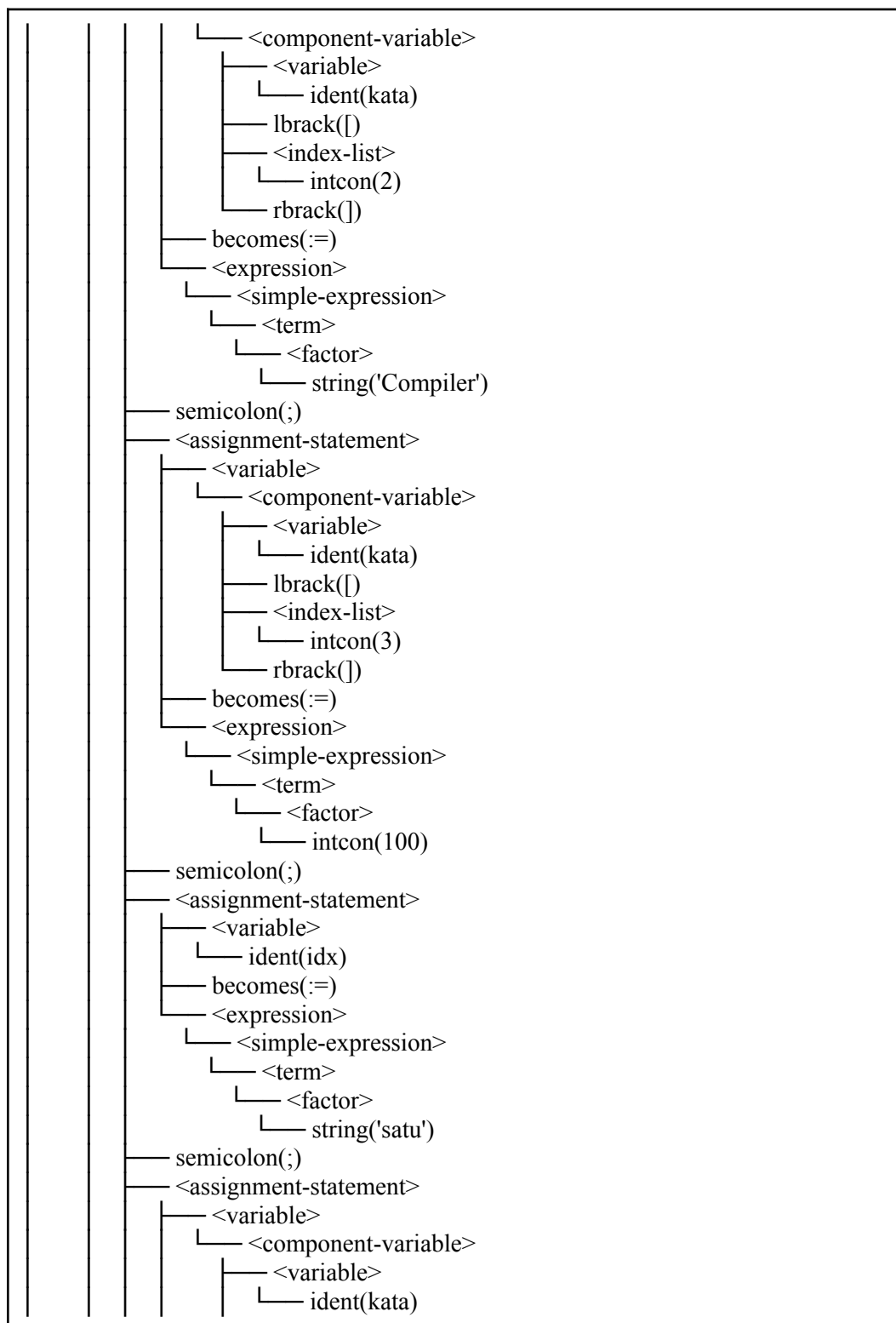
idx	xtyp	etyp	eref	low	high	elsz	size
1	integer	integer	0	1	5	1	5

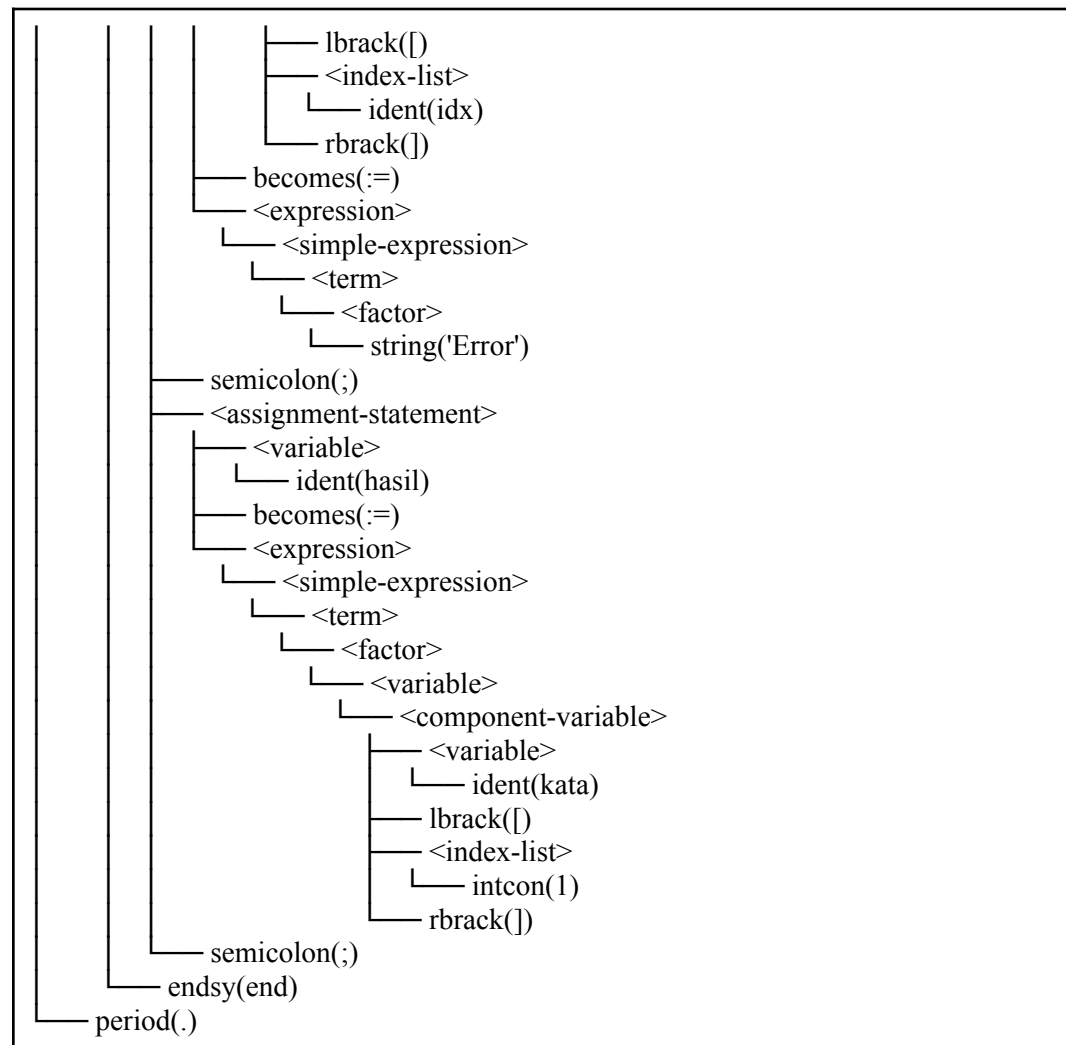
7. Test Case 7

7.1. Input:









7.2. Output:

--- Semantic Errors (3) ---

- Semantic Error: Type mismatch in assignment: cannot assign 'integer' to 'string'
- Semantic Error: Array index must be of Integer type
- Semantic Error: Type mismatch in assignment: cannot assign 'string' to 'integer'

--- Symbol Table (tab) ---

idx	identifier	obj	type	ref	nrm	lev	adr	link
38	testarrayerror	program	none	0	1	0	0	37



39	kata	variable	array	1	1	1	0	0
40	idx	variable	string	0	1	1	3	39
41	hasil	variable	integer	0	1	1	4	40

--- Block Table (btab) ---

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	38	0	0	0
---	----	---	---	---

1	41	0	0	5
---	----	---	---	---

--- Array Table (atab) ---

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

1	integer	string	0	1	3	1	3
---	---------	--------	---	---	---	---	---



BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada milestone 3 ini, compiler Arion berhasil dikembangkan hingga tahap semantic analysis. Program mampu melakukan konversi parse tree menjadi *Abstract Syntax Tree* (AST), membangun simbol pada tabel, melakukan pengecekan batasan atau scope, serta melakukan validasi tipe data pada expression dan assignment. Selain itu, compiler juga akan melakukan pengecekan atau scan sehingga dapat mendeteksi berbagai semantic error seperti *undeclared identifier*, *type mismatch*, dan kesalahan dalam menggunakan operator. Dengan adanya *semantic analysis*, validitas program dapat diperiksa secara lebih mendalam sebelum memasuki tahap code generation

2. Saran

Meskipun program telah berjalan dengan baik sesuai spesifikasi, masih terdapat beberapa hal yang dapat ditingkatkan ke depannya. Pertama, penanganan error saat ini sudah dapat mendeteksi dan melaporkan semantic error, namun pesan error yang ditampilkan masih bisa lebih spesifik, misalnya dengan menyertakan nomor baris tempat error terjadi sehingga pengguna lebih mudah melacak letak kesalahan pada source code.

Selain itu, program saat ini belum melakukan pengecekan apakah variabel sudah diinisialisasi sebelum digunakan. Hal ini dapat menjadi sumber bug yang sulit dilacak, sehingga penambahan validasi tersebut akan meningkatkan keandalan semantic analyzer secara keseluruhan.

Dari sisi manajemen memori, alokasi node pada program masih menggunakan raw pointer, yang berpotensi menimbulkan memory leak terutama pada program Arion yang lebih besar dan kompleks. Penggunaan smart pointer seperti `std::unique_ptr` dapat menjadi solusi yang lebih aman untuk kedepannya.



Terakhir, validasi argumen saat pemanggilan prosedur dan fungsi masih belum dilakukan secara menyeluruh. Pengecekan jumlah dan kesesuaian tipe argumen aktual terhadap parameter formal akan menjadi tambahan yang berguna, terutama dalam persiapan menghadapi tahap Intermediate Code Generation pada milestone berikutnya.

BAB V

LAMPIRAN

1. Link Repository GitHub

Berikut adalah tautan untuk mengakses repository pada platform GitHub:

<https://github.com/Alvacodee/KOI-Tubes-IF2224-2026>

2. Link Workspace Diagram Transisi DFA

Berikut adalah tautan untuk mengakses workspace pada platform draw.io:

<https://drive.google.com/file/d/1RiAAAnoB2hjkNPZm29Pn6T1FA5qK6AiLD/view?usp=sharing>

3. Pembagian Tugas

NAMA	NIM	PEMBAGIAN TUGAS	Persentase Kontribusi
Zahran Alvan Putra Winarko	13524124	Manajemen Symbol Table & Deklarasi	25%
Neswa Eka Anggara	13524136	Control Flow Validation & Error Handling	25%



Daniel Putra Rywandi S.	13524143	Infrastruktur AST & I/O	25%
Muh. Hartawan Haidir	13524147	Type Checking & Evaluasi Ekspresiment	25%



BAB VI

REFERENSI

1. Edunex ITB. (n.d.). *Formal Language and Automata Theory Materials*. Diakses pada 7 Mei 2026, dari <https://edunex.itb.ac.id/>